ESRG

Embedded Systems Research Group

CENTROALGORITMI

Secure Asset Guard

João Mendes– PG60206
Tiago Oliveira– PG60217

Universidade do Minho
Escola de Engenharia

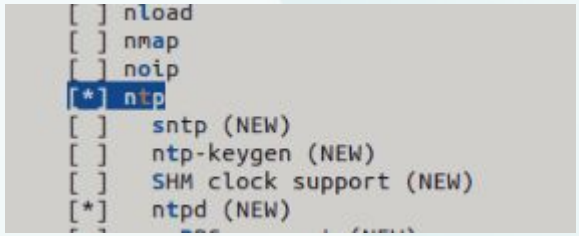
COMPETE

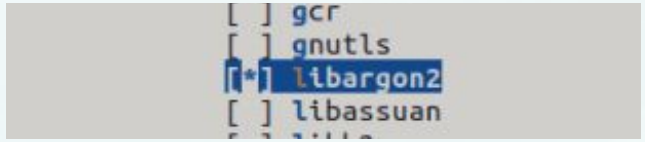
QREN Quadro de Referência Estratégico Nacional

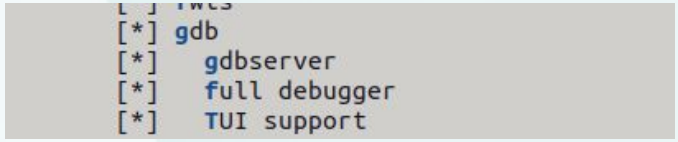
União Europeia
Fundo Europeu de Desenvolvimento Regional

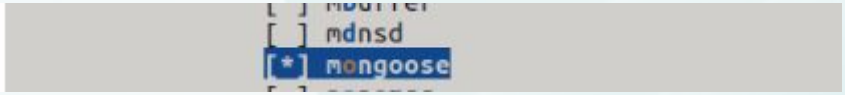
FCT Fundação para a Ciência e a Tecnologia
Ministério da Ciência, Tecnologia e Ensino Superior

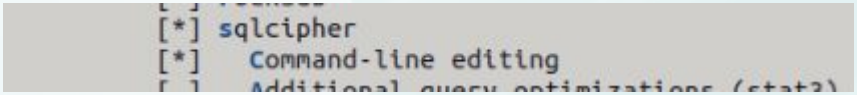
Buildroot libraries

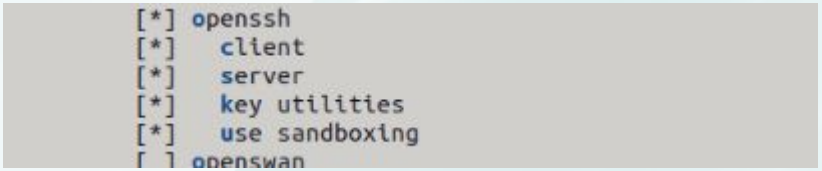




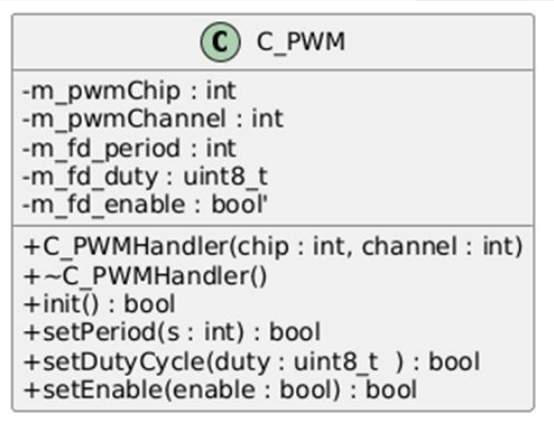








PWM




```cpp
class C_PWM
{
private:
    int m_pwmChip;
    int m_pwmChannel;
    int m_fd_period;
    uint8_t m_fd_duty;
    bool m_fd_enable;
public:
    // Constructor
    C_PWM(int chip, int channel);
    // Destructor
    ~C_PWM();
    // inicialization
    bool init();
    // period
    bool setPeriodns(int s);
    //duty cyle percentage
    bool setDutyCycle(uint8_t duty);
    // activates/deactivates pwm
    bool setEnable(bool enable);
};
```

Device tree PWM overlay


```
# cd /sys/class/pwm/
# ls
#
```

dtoverlay=pwm,pin=12,func=4

```
# cd /sys/class/pwm/
# ls
pwmchip0
#
```

BCM2711 Datasheet

Bits	Name	Description	Type	Reset
31:30	Reserved.	-	-	-
29:27	FSEL9	FSEL9 - Function Select 9 000 = GPIO Pin 9 is an input 001 = GPIO Pin 9 is an output 100 = GPIO Pin 9 takes alternate function 0 101 = GPIO Pin 9 takes alternate function 1 110 = GPIO Pin 9 takes alternate function 2 111 = GPIO Pin 9 takes alternate function 3 011 = GPIO Pin 9 takes alternate function 4 010 = GPIO Pin 9 takes alternate function 5	RW	0x0
26:24	FSEL8	FSEL8 - Function Select 8	RW	0x0
23:21	FSEL7	FSEL7 - Function Select 7	RW	0x0
20:18	FSEL6	FSEL6 - Function Select 6	RW	0x0
17:15	FSEL5	FSEL5 - Function Select 5	RW	0x0
14:12	FSEL4	FSEL4 - Function Select 4	RW	0x0
11:9	FSEL3	FSEL3 - Function Select 3	RW	0x0
8:6	FSEL2	FSEL2 - Function Select 2	RW	0x0
5:3	FSEL1	FSEL1 - Function Select 1	RW	0x0
2:0	FSEL0	FSEL0 - Function Select 0	RW	0x0

5.1.2 GPIO Alternate Functions

GPIO	Default Pull	ALT0	ALT1	ALT2	ALT3	ALT4	ALT5
0	High	SDA0	SA5	PCLK	SPI3_CE0_N	TXD2	SDA6
1	High	SCL0	SA4	DE	SPI3_MISO	RXD2	SCL6
2	High	SDA1	SA3	LCD_VSYNC	SPI3_MOSI	CTS2	SDA3
3	High	SCL1	SA2	LCD_HSYNC	SPI3_SCLK	RTS2	SCL3
4	High	GPCLK0	SA1	DPI_D0	SPI4_CE0_N	TXD3	SDA3
5	High	GPCLK1	SA0	DPI_D1	SPI4_MISO	RXD3	SCL3
6	High	GPCLK2	SOE_N	DPI_D2	SPI4_MOSI	CTS3	SDA4
7	High	SPI0_CE1_N	SWE_N	DPI_D3	SPI4_SCLK	RTS3	SCL4
8	High	SPI0_CE0_N	SD0	DPI_D4	-	TXD4	SDA4
9	Low	SPI0_MISO	SD1	DPI_D5	-	RXD4	SCL4
10	Low	SPI0_MOSI	SD2	DPI_D6	-	CTS4	SDA5
11	Low	SPI0_SCLK	SD3	DPI_D7	-	RTS4	SCL5
12	Low	PWM0	SD4	DPI_D8	SPI5_CE0_N	TXD5	SDA5
13	Low	PWM1	SD5	DPI_D9	SPI5_MISO	RXD5	SCL5

CENTROALGORITMI

```
# cd /sys/class/pwm/pwmchip0/
# ls
device      export      npwm        power       subsystem   uevent      unexport
# echo 0 > export
# ls
device      export      npwm        power       pwm0        subsystem   uevent      unexport
```

PWM Channel 0 Enabled

```
bool C_PWM::init()
{
    string exportPath = "/sys/class/pwm/pwmchip" + to_string(m_pwmChip)+ "/export";

    int fd = open( file: exportPath.c_str(), oflag: O_WRONLY);
    if (fd < 0)
    {
        cerr << "Erro ao abrir export\n";
        return false;
    }

    string channel = to_string(m_pwmChannel);
    if (write(fd, buf: channel.c_str(), n: channel.size()) < 0)
    {
        cerr << "Erro ao exportar PWM\n";
        close(fd);
        return false;
    }

    close(fd);
    return true;
}
```

```
int open(const char *pathname, int flags, mode_t mode);
```

- open() returns -1 if not successful and returns file descriptor if successful

```
ssize_t write(int fd, const void *buf, size_t count);
```

- write() returns -1 if not successful and returns number of bytes written if successful

CENTROALGORITMI

```
# cd /sys/class/pwm/pwmchip0/pwm0/
# ls
capture     duty_cycle  enable      period      polarity    power       uevent
```

Linux is expecting nanoseconds in Period and Duty_Cycle and 0 or 1 in Enable

```cpp
bool C_PWM::setDutyCycle(uint8_t duty) {
    m_fd_duty = duty;
    int dutyns = (m_fd_period * duty) / 100;

    // Caminho para o ficheiro duty_cycle
    string dutyPath =
            "/sys/class/pwm/pwmchip" + to_string(m_pwmChip) +
            "/pwm" + to_string(m_pwmChannel) + "/duty_cycle";
    int fd = open( file: dutyPath.c_str(),  oflag: O_WRONLY);
    if (fd < 0) {
        cerr << "Erro ao abrir duty_cycle\n";
        return false;
    }
    string val = to_string(dutyns);
    if (write(fd,  buf: val.c_str(),  n: val.size()) < 0) {
        cerr << "Erro ao escrever duty_cycle\n";
        close(fd);
        return false;
    }
    close(fd);
    return true;
}
```

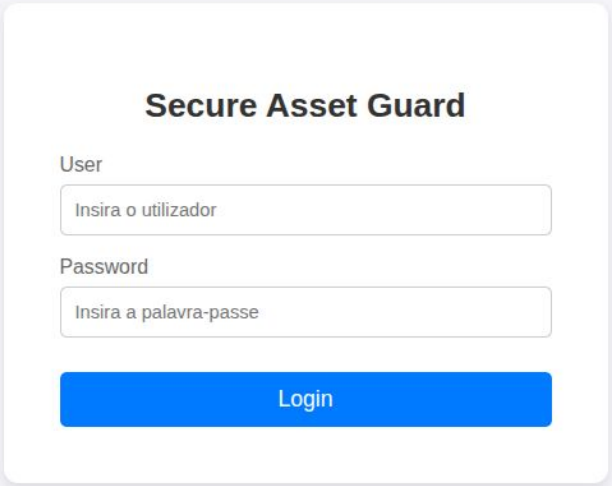
```cpp
bool C_PWM::setPeriodns(int s) {
    m_fd_period = s;
    string periodPath =
            "/sys/class/pwm/pwmchip" + to_string(m_pwmChip) +
            "/pwm" + to_string(m_pwmChannel) + "/period";
    int fd = open( file: periodPath.c_str(),  oflag: O_WRONLY);
    if (fd < 0)
    {
        cerr << "Erro ao abrir period\n";
        return false;
    }
    string val = to_string(s);
    if (write(fd,  buf: val.c_str(),  n: val.size()) < 0) {
        cerr << "Erro ao escrever period\n";
        close(fd);
        return false;
    }
    close(fd);
    return true;
}
```

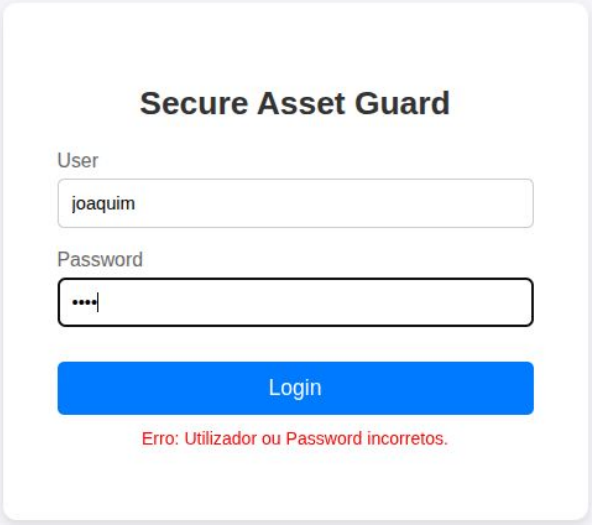
```cpp
bool C_PWM::setEnable(bool enable)
{
    m_fd_enable = enable;
    string enablePath =
            "/sys/class/pwm/pwmchip" + to_string(m_pwmChip) +
            "/pwm" + to_string(m_pwmChannel) + "/enable";

    int fd = open( file: enablePath.c_str(),  oflag: O_WRONLY);
    if (fd < 0)
    {
        cerr << "Erro ao abrir enable\n";
        return false;
    }
    string val = enable ? "1" : "0";
    if (write(fd,  buf: val.c_str(),  n: val.size()) < 0)
    {
        cerr << "Erro ao escrever enable\n";
        close(fd);
        return false;
    }
    close(fd);
    return true;
}
```

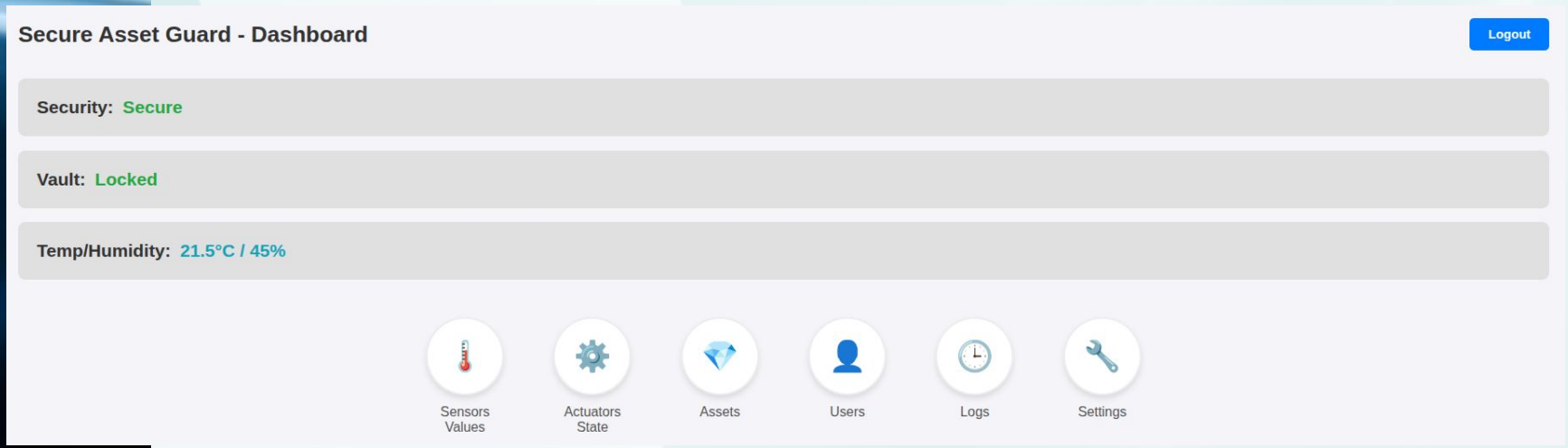
ESRG
Embedded Systems
Research Group

Interface

CENTROALGORITMI







Device tree UART overlay


```
# cd /sys/class/tty
# ls
```

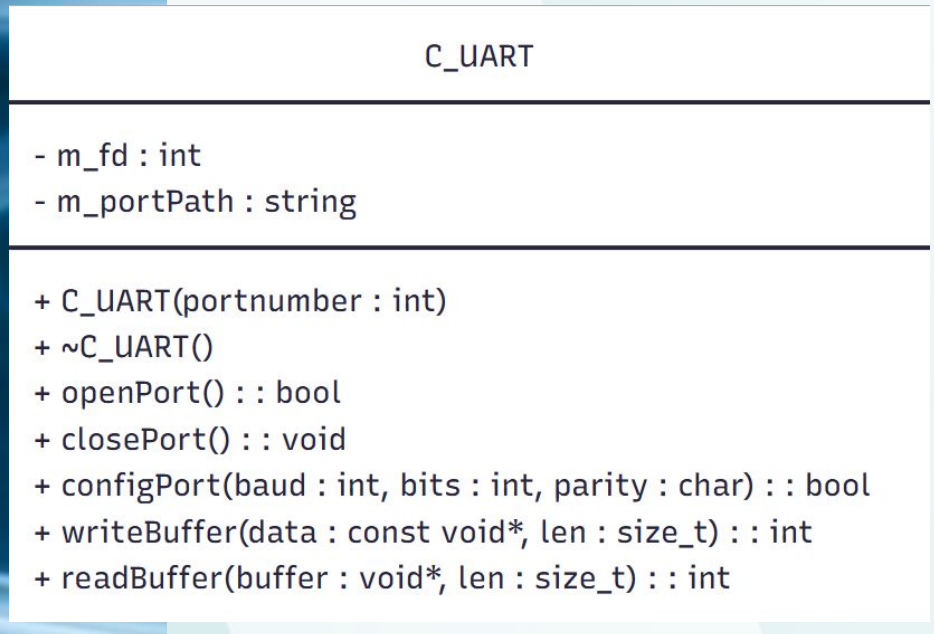
```
dtoverlay=uart2
```

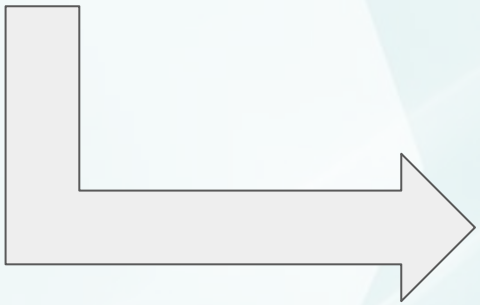
```
# cd /dev/
# ls
ttyAMA2
```

```
# cd /sys/class/tty
# ls
ttyAMA2
```

GPIO	Default Pull	ALT0	ALT1	ALT2	ALT3	ALT4	ALT5
0	High	SDA0	SA5	PCLK	SPI3_CE0_N	TXD2	SDA6
1	High	SCL0	SA4	DE	SPI3_MISO	RXD2	SCL6

Class UART





```
class C_UART {

    int m_fd;
    string m_portPath;
public:
    C_UART(int portnumber);

    // Destrutor
    ~C_UART();

    bool openPort();

    void closePort();

    bool configPort(int baud, int bits, char parity);

    int writeBuffer(const void* data, size_t len);

    int readBuffer(void* buffer, size_t len);

};
```

Device tree I2C overlay


```
# cd /sys/class/i2c-adapter
# ls
#
```

```
#i2c1 Sda and Scl mapped by predefinition to GPIO 2 and 3
dtoverlay=i2c1
```

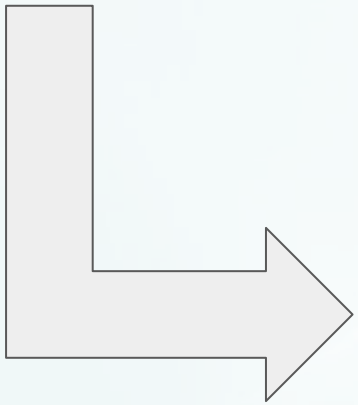
```
# cd /sys/class/i2c-adapter
# ls
i2c-1
```

```
# cd /dev/
# ls
autofs
bus
cachefiles
console
cpu_dma_latency
dma_heap
fd
full
gpiochip0
gpiochip1
hwrng
i2c-1
```

GPIO	Default Pull	ALT0	ALT1	ALT2	ALT3	ALT4	ALT5
2	High	SDA1	SA3	LCD_VSYNC	SPI3_MOSI	CTS2	SDA3
3	High	SCL1	SA2	LCD_HSYNC	SPI3_SCLK	RTS2	SCL3

Class I2C

C_I2C
- m_slaveaddress : uint8_t - m_fd : int - m_devicePath : string
+ C_I2C(i2cbusnum : int, slave_address : uint8_t) + ~C_I2C() + init() : : bool + closeI2C() : : void + writeRegister(reg : uint8_t, value : uint8_t) : : bool + readRegister(reg : uint8_t, value : uint8_t*) : : bool + readBytes(reg : uint8_t, buffer : uint8_t*, len : size_t) : : bool



```
class C_I2C {
    uint8_t m_slaveaddress;
    int m_fd;
    string m_devicePath;
public:
    C_I2C(int i2cbusnum, uint8_t slave_address);
    ~C_I2C();
    //open dev directory and "set" slave address
    bool init();
    //close fd
    void closeI2C();
    //used mainly to write some configuration in sensor specific register
    bool writeRegister(uint8_t reg, uint8_t value);
    //used to read only one sensor specific reg
    bool readRegister(uint8_t reg, uint8_t& value);
    //used to read more than one reg
    bool readBytes(uint8_t reg, uint8_t* buffer, size_t len);

};
```

CENTROALGORITMI

```
C_I2C::C_I2C(int i2cbusnum, uint8_t slave_address)
    : m_slaveaddress(slave_address), m_fd(-1)
{

    m_devicePath = "/dev/i2c-" + to_string(i2cbusnum);
}


C_I2C::~C_I2C() {
    closeI2C();
}

bool C_I2C::init() {
    m_fd = open( file: m_devicePath.c_str(), oflag: O_RDWR);
    if (m_fd < 0) {
        cerr << "C_I2C: Erro ao abrir dev/...\n";
        return false;
    }

    if (ioctl(m_fd, request: I2C_SLAVE, m_slaveaddress) < 0) {
        cerr<<"C_I2C: Erro ao definir slave (ioctl)\n";
        return false;
    }

    return true;
}
```

ioctl() to define the slave address that i2c driver will be using in the beginning of write() and read()

Class GPIO

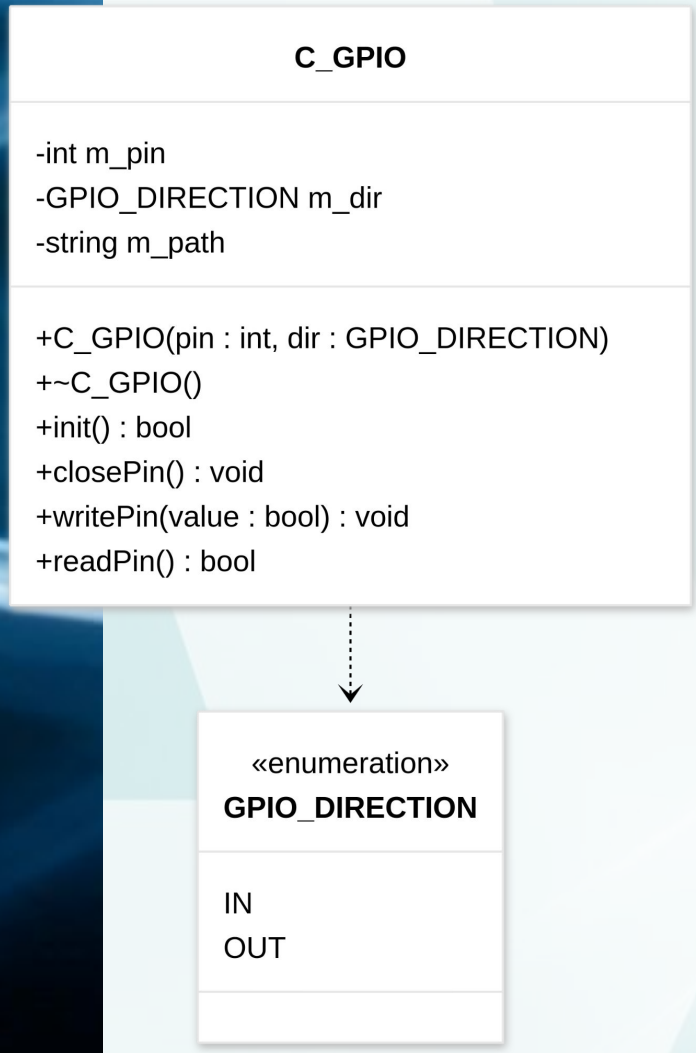
- sysfs for GPIO access

```
# cd /sys/class/gpio
# ls
export       gpiochip512  gpiochip570  unexport
# cd gpiochip512
# ls
base       device     label      ngpio      power      subsystem  uevent
# cat base
512
# cat label
pinctrl-bcm2711
#
```

Linux GPIO controller has a base of 512

```
# cd /sys/class/gpio
# echo 530 > export
# ls
export       gpio530      gpiochip512  gpiochip570  unexport
# cd gpio530
# ls
active_low  device     direction  edge       power      subsystem  uevent     value
#
```

530-512= GPIO Pin 18




```cpp
enum GPIO_DIRECTION { IN, OUT };

class C_GPIO {
public:

    C_GPIO(int pin, GPIO_DIRECTION dir);

    ~C_GPIO();

    //Pin export
    bool init();

    //Pin unexport
    void closePin();

    //output value
    void writePin(bool value);

    //read input value
    bool readPin();

private:
    int m_pin;
    GPIO_DIRECTION m_dir;
    string m_path;//sysfs path
};
```

CENTROALGORITMI

```
#define GPIO_BASE 512
```

```
C_GPIO::C_GPIO(int pin, GPIO_DIRECTION dir)
    : m_dir(dir)
{
    m_pin=pin+GPIO_BASE;
    m_path = "/sys/class/gpio/gpio" + to_string(m_pin);
}
```

GPIO Interrupt Device Driver

Creating a device driver that will be sending signals to our main process when there is a change in a specific pin value

ioctl()- used to define the PID to where the signals are sent
write()-used to activate and deactivate interrupts
open()
close()

GPIO


```
#define BCM2708_PERI_BASE       0xfe000000
#define GPIO_BASE (BCM2708_PERI_BASE + 0x200000) // GPIO controller

struct GpioRegisters
{

    uint32_t GPFSEL[6];

    uint32_t ReservedGap[51];

    uint32_t PUP_PDN[4];
};
```

5.2. Register View

The GPIO has the following registers. All accesses are assumed to be 32-bit. The GPIO register base address is 0x7e200000.

Offset	Name	Description
0x00	GPFSEL0	GPIO Function Select 0
0x04	GPFSEL1	GPIO Function Select 1
0x08	GPFSEL2	GPIO Function Select 2
0x0c	GPFSEL3	GPIO Function Select 3
0x10	GPFSEL4	GPIO Function Select 4
0x14	GPFSEL5	GPIO Function Select 5

Module init


```
// 1. Alocar Major/Minor
if ((ret = alloc_chrdev_region(&irqDevice_majorminor, 0, NUM_IRQS, DEVICE_NAME)) < 0) {
    return ret;
}
```

```
// 2. Create class
if (IS_ERR(irqDevice_class = class_create(CLASS_NAME))) {
    unregister_chrdev_region(irqDevice_majorminor, NUM_IRQS);
    return PTR_ERR(irqDevice_class);
}
```

```
// 3.add cdev and associate file operation struct witrh major and each minor number
cdev_init(&c_dev, &irqDevice_fops);
c_dev.owner = THIS_MODULE;

if ((ret = cdev_add(&c_dev, irqDevice_majorminor, NUM_IRQS)) < 0) {
```

```
// 4. Create /dev/irqx files that we can access directly
for (i = 0; i < NUM_IRQS; i++) {
    dev_ret = device_create(irqDevice_class, NULL,
                            MKDEV(MAJOR(irqDevice_majorminor), i),
                            NULL, "irq%d", i);
```

```
// 5.ioremap to make the physical GPIO registers accessible to the kernel
s_pGpioRegisters = (struct GpioRegisters *)ioremap(GPIO_BASE, sizeof(struct GpioRegisters));
```

ESRG

Embedded Systems Research Group

```c
struct SensorConfig {
    int gpio_pin;
    unsigned long trigger;
    int signal_id;
    char *name;
    bool enabled;
    unsigned long last_time;
};

static struct SensorConfig my_sensors[] = {
    {23, IRQF_TRIGGER_RISING,  44, "reed_switch", true,0},
    {27, IRQF_TRIGGER_RISING,  45, "pir_sensor",  true,0},
    {22, IRQF_TRIGGER_RISING,  46, "fingerprint", true,0},
    { 5, IRQF_TRIGGER_FALLING, 47, "rfid_reader", true,0}
};
```

```c
// 6. Configure interruption
for (i = 0; i < NUM_IRQS; i++) {
    struct SensorConfig *sensor = &my_sensors[i];
    SetGPIOFunction(s_pGpioRegisters, sensor->gpio_pin, 0);
```

```c
//irq number associated to that gpio pin
gpio_logical = 512 + sensor->gpio_pin;
irq_numbers[i] = gpio_to_irq(gpio_logical);
```

```c
//request irq number to the irq number
ret = request_irq(irq_numbers[i],gpio_irq_handler, sensor->trigger, sensor->name, (void *)sensor);
```

Passing the information from each sensor to the IRQ handler so we can identify which sensor it is

gpio_irq_handler()


```c
static irqreturn_t gpio_irq_handler(int irq, void *dev_id) {

    struct kernel_siginfo info;
    struct SensorConfig *sensor_atual = (struct SensorConfig *)dev_id;

    int pino = sensor_atual->gpio_pin;
    int sinal = sensor_atual->signal_id;


    unsigned long now = jiffies;

    if ((now - sensor_atual->last_time) < msecs_to_jiffies(500)) {
        return IRQ_HANDLED;
    }

    // if the time has passed store the time of new sensor interrupt signal
    sensor_atual->last_time = now;

    if (task != NULL) {
        memset(&info, 0, sizeof(struct kernel_siginfo));
        info.si_signo = sinal;
        info.si_code = SI_QUEUE;
        info.si_int = pino;

        if (send_sig_info(sinal, &info, task) < 0) {
            pr_err("irqModule: Erro ao enviar sinal.\n");
        }
    }
```

Restore sensor specific information

Add debounce to guarantee we don't send a lot of signals in a row

Send signal to our process whose PID Is defined on "task"

System calls


```
// FILE OPERATIONS
static struct file_operations irqDevice_fops = {
    .owner          = THIS_MODULE,
    .open           = irq_device_open,
    .release        = irq_device_close,
    .write          = irq_device_write,     // A
    .read           = NULL,                 // N
    .unlocked_ioctl = irq_device_ioctl,
};
```

```
static long irq_device_ioctl(struct file *file, unsigned int cmd, unsigned long arg) {

    if (cmd == REGIST_PID) {
        //get PID
        task = get_current();
```

```
static int irq_device_open(struct inode* p_inode, struct file *p_file) {

    int minor = iminor(p_inode);

    if (minor >= NUM_IRQS) return -ENODEV;

    // store minor number so we can access to the specific sensor struct in write()
    p_file->private_data = (void *)(unsigned long)minor;
```

```c
static ssize_t irq_device_write(struct file *pfile, const char __user *pbuff, size_t len, loff_t *off) {
    char command;
    unsigned long minor;
    struct SensorConfig *sensor;

    // 1. recover which sensor is(stored in opne())
    minor = (unsigned long)pfile->private_data;
    sensor = &my_sensors[minor]; // Apontar para a configuração correta

    // 2.safely copy user commnad that comes form user space
    if (copy_from_user(&command, pbuff, 1)) return -EFAULT;

    pr_alert("irqModule: Comando '%c' recebido para %s (IRQ %d)\n",
             command, sensor->name, irq_numbers[minor]);

    //3. enable and disable interrupt
    if (command == '0') {
        // Only disable if it is enabled
        if (sensor->enabled == true) {
            disable_irq(irq_numbers[minor]); // disable irq of speific gpio
            sensor->enabled = false;
            pr_alert("irqModule: Interrupção DESATIVADA para %s\n", sensor->name);
        }
    }
    else if (command == '1') {
        // Only enable if it is desabled
        if (sensor->enabled == false) {
            enable_irq(irq_numbers[minor]); //enable irq of specific pin
            sensor->enabled = true;
            pr_alert("irqModule: Interrupção ATIVADA para %s\n", sensor->name);
        }
    }
```

So a peripheral described in this document as being at legacy address 0x7Enn_nnnn is available in the 35-bit addres space at 0x4_7Enn_nnnn, and visible to the ARM at 0x0_FEnn_nnnn if Low Peripheral mode is enabled.

The GPIO has the following registers. All accesses are assumed to be 32-bit. The GPIO register base address is `0x7e200000`.

Offset	Name	Description
0x00	GPFSEL0	GPIO Function Select 0
0x04	GPFSEL1	GPIO Function Select 1
0x08	GPFSEL2	GPIO Function Select 2
0x0c	GPFSEL3	GPIO Function Select 3
0x10	GPFSEL4	GPIO Function Select 4
0x14	GPFSEL5	GPIO Function Select 5
0x1c	GPSET0	GPIO Pin Output Set 0
0x20	GPSET1	GPIO Pin Output Set 1
0x28	GPCLR0	GPIO Pin Output Clear 0
0x2c	GPCLR1	GPIO Pin Output Clear 1

ESRG

Embedded Systems Research Group

Shared types

CENTROALGORITMI

Defines thread priority

```
// ============================================
// thread priority
// ============================================
enum ThreadPriority_enum : int {
    PRIO_LOW        = 10,
    PRIO_MEDIUM     = 30,
    PRIO_HIGH       = 99
};

// ============================================
// IDs of actuators
// ============================================
enum ActuatorID_enum : uint8_t {
    ID_SERVO_ROOM = 0,
    ID_SERVO_VAULT = 1,
    ID_FAN = 2,
    ID_ALARM_ACTUATOR = 3,
    ID_ACTUATOR_COUNT = 4
};

// ============================================
// IDs of Sensors
// ============================================
enum SensorID_enum : uint8_t {
    ID_SHT31 = 0,
    ID_RDM6300 = 1,
    ID_YRM1001 = 2,
    ID_FINGERPRINT = 3,
    ID_SENSOR_COUNT = 4
};
```

```
// ============================================
// Types of Logs
// ============================================
enum LogType_enum : uint8_t {
    LOG_TYPE_ACTUATOR = 0,
    LOG_TYPE_SENSOR = 1,
    LOG_TYPE_ACCESS = 2,
    LOG_TYPE_SYSTEM = 3,
    LOG_TYPE_ALERT = 4,
    LOG_TYPE_INVENTORY = 5
};

// ============================================
// commands for actuator
// ============================================
struct ActuatorCmd {
    ActuatorID_enum actuatorID;
    uint8_t value;
    // 0 = PWM OFF/LIVRE (Destrancar)
    // > 0 = Ângulo/PWM ON (Trancar)

    ActuatorCmd() : actuatorID(ID_SERVO_ROOM), value(0) {}
    ActuatorCmd(ActuatorID_enum id, uint8_t val)
        : actuatorID(id), value(val) {}
};
```

It categorizes events before they are sent to the database.

commands for the thread tAct

```
struct DatabaseLog {
    LogType_enum logType;
    uint32_t entityID;
    double value;
    double value2;
    uint32_t timestamp;
    char description[266]{};
    DatabaseLog()
        : logType(LOG_TYPE_SYSTEM),
          entityID(0),
          value(0.0),
          value2(0.0),
          timestamp(0) {
        description[0] = '\0';
    }
};
```

Universal event container for logging sensor data, user access, and system alerts

CENTROALGORITMI

```
enum e_DbCommand {
    DB_CMD_ENTER_ROOM_RFID,
    DB_CMD_LEAVE_ROOM_RFID,
    DB_CMD_UPDATE_ASSET,
    DB_CMD_USER_IN_PIR,
    DB_CMD_WRITE_LOG,
    DB_CMD_LOGIN,
    DB_CMD_GET_DASHBOARD,
    DB_CMD_GET_SENSORS,
    DB_CMD_GET_ACTUATORS,
    DB_CMD_ADD_USER,
    DB_CMD_DELETE_USER,
    DB_CMD_UPDATE_TEMP_THRESHOLD,
    DB_CMD_UPDATE_SAMPLING_TIME,
    DB_CMD_REGISTER_USER,
    DB_CMD_GET_USERS,
    DB_CMD_CREATE_USER,
    DB_CMD_MODIFY_USER,
    DB_CMD_REMOVE_USER,
    DB_CMD_GET_ASSETS,
    DB_CMD_CREATE_ASSET,
    DB_CMD_MODIFY_ASSET,
    DB_CMD_REMOVE_ASSET,
    DB_CMD_GET_SETTINGS,
    DB_CMD_GET_SETTINGS_THREAD,
    DB_CMD_UPDATE_SETTINGS,
    DB_CMD_FILTER_LOGS,
    DB_CMD_STOP_ENV_SENSOR
};
```

Standardized command set for structured database communication and request handling via IPC

```
struct DatabaseMsg {
    e_DbCommand command;
    union {
        char rfid[11];
        DatabaseLog log;
        Data_RFID_Inventory rfidInventory;
        LoginRequest login;
        UserData user;
        AssetData asset;
        SystemSettings settings;
        LogFilter logFilter;
        uint32_t userId;
    } payload;
};
```

Universal request bridge: all system components to Database

```
struct AuthResponse {
    e_DbCommand command;

    union {
        struct {
            bool authorized;
            uint32_t userId;
            uint32_t accessLevel;
        } auth;

        SystemSettings settings;
    } payload;
};
```

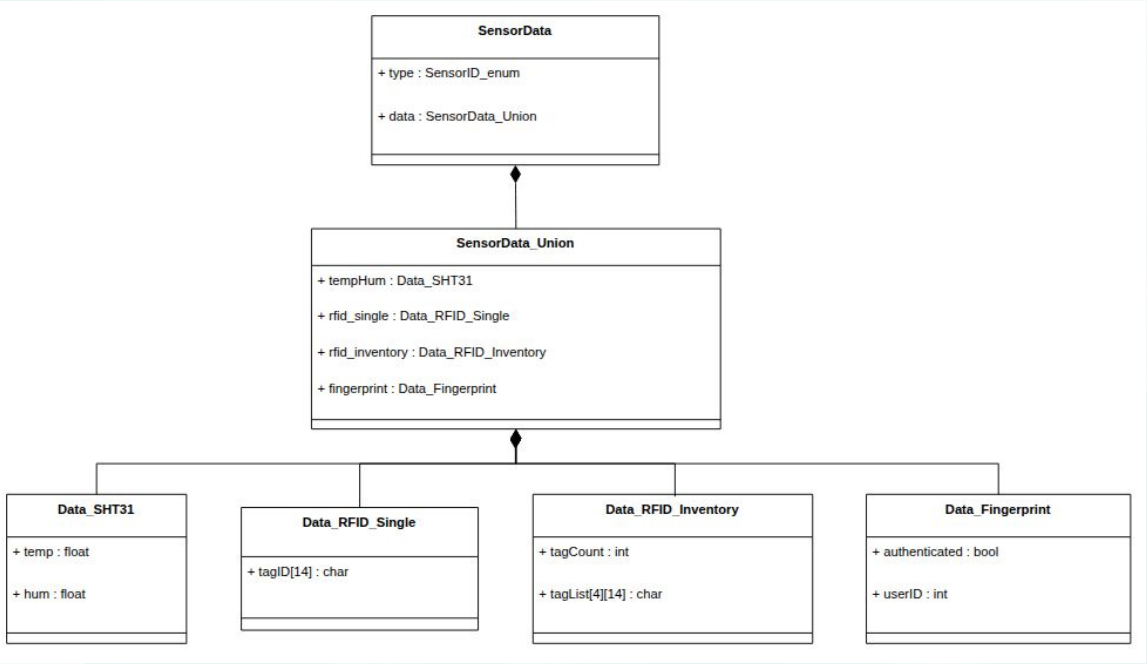
DB-to-Threads: Authentication and sync response

```
struct DbWebResponse {
    bool success;
    char jsonData[8192];
    char errorMsg[256];
};
```

DB-to-Web: JSON data transport for UI.

CENTROALGORITMI

```c
#ifndef C_SENSOR_H
#define C_SENSOR_H
#include "SharedTypes.h"

// Sensor-specific Data Structures
struct Data_SHT31 {
    float temp;
    float hum;
};

struct Data_RFID_Single {
    char tagID[14];
};

struct Data_RFID_Inventory {
    int tagCount;
    char tagList[4][14];
};

struct Data_Fingerprint {
    bool authenticated;
    int userID;
};

union SensorData_Union {
    Data_SHT31 tempHum;
    Data_RFID_Single rfid_single;
    Data_RFID_Inventory rfid_inventory;
    Data_Fingerprint fingerprint;
};

struct SensorData {
    SensorID_enum type;
    SensorData_Union data;
};
```



Class C_Sensor


```
class C_Sensor {
protected:
    SensorID_enum m_sensorID;

public:
    C_Sensor(SensorID_enum id) : m_sensorID(id) {}

    virtual ~C_Sensor();

    virtual bool init() = 0;
    virtual bool read(SensorData* data) = 0;

    SensorID_enum get_ID() const { return m_sensorID; }
};

#endif // C_SENSOR_H
```

Ensures cleanup of derived sensor resources even when using a base pointer

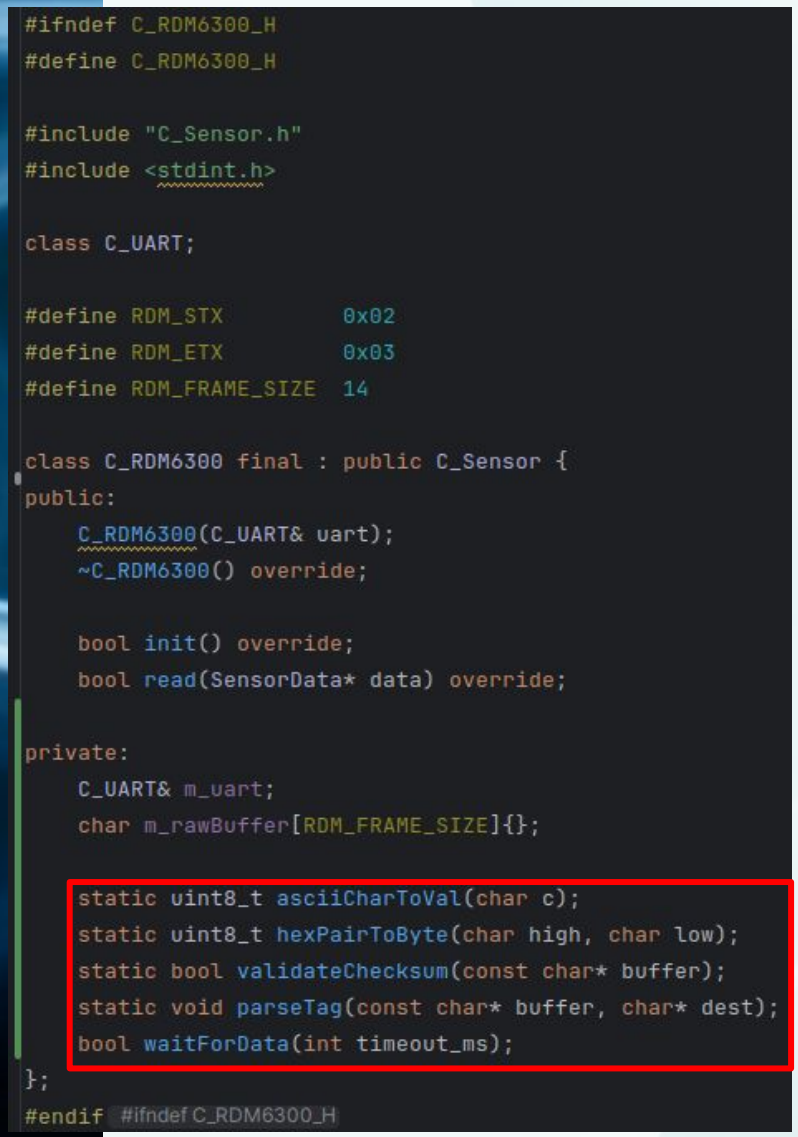
Purely Virtual Methods

C_Sensor
m_sensorID : SensorID_enum
+ C_Sensor(id : SensorID_enum) + virtual ~C_Sensor() *+ virtual init() : bool* *+ virtual read(data : SensorData*) : bool* + get_ID() : : SensorID_enum

Class C_RDM6300

CENTROALGORITMI

```cpp
#ifndef C_RDM6300_H
#define C_RDM6300_H

#include "C_Sensor.h"
#include <stdint.h>

class C_UART;

#define RDM_STX         0x02
#define RDM_ETX         0x03
#define RDM_FRAME_SIZE  14

class C_RDM6300 final : public C_Sensor {
public:
    C_RDM6300(C_UART& uart);
    ~C_RDM6300() override;

    bool init() override;
    bool read(SensorData* data) override;

private:
    C_UART& m_uart;
    char m_rawBuffer[RDM_FRAME_SIZE]{};

    static uint8_t asciiCharToVal(char c);
    static uint8_t hexPairToByte(char high, char low);
    static bool validateChecksum(const char* buffer);
    static void parseTag(const char* buffer, char* dest);
    bool waitForData(int timeout_ms);
};
#endif #ifndef C_RDM6300_H
```



Internal helpers for protocol validation and data conversion.

```cpp
#include "C_RDM6300.h"
#include "C_UART.h"
#include <iostream>
#include <cstring>
#include <poll.h>
#include <unistd.h>

C_RDM6300::C_RDM6300(C_UART& uart)
    : C_Sensor(ID_RDM6300), m_uart(uart) {}

C_RDM6300::~C_RDM6300() = default;

bool C_RDM6300::init() {
    if (!m_uart.openPort()) return false;
    // RDM6300 default: 9600 baud, 8 data bits, no parity, 1 stop bit
    if (!m_uart.configPort( baud: 9600,  bits: 8,  parity: 'N')) return false;
    return true;
}

// Wait for data with timeout using poll()
// Returns true if data available, false on timeout
bool C_RDM6300::waitForData(int timeout_ms) const {
    struct pollfd pfd;
    pfd.fd = m_uart.getFd();
    pfd.events = POLLIN;     // Wait for read-ready

    // poll() blocks without CPU usage
    int ret = poll(&pfd,  nfds: 1, timeout_ms);

    return (ret > 0);        // >0 = data ready, 0 = timeout, <0 = error
}
```

Configuration of the UART to a baudrate of 9600

This function implements an event-driven architecture using POSIX `poll`. The thread suspends its execution and waits for a kernel notification, waking up strictly when the RFID module transmits data. This mechanism ensures precise timing management for the communication protocol without the need for continuous software checks.

CENTROALGORITMI

```
bool C_RDM6300::read(SensorData* data) {
    /* Wait for packet start (STX byte) with 1s timeout.
       If no card present, returns quickly allowing thread exit checks. */
    if (!waitForData( timeout_ms: 1000)) {
        return false;
    }
    // Read STX byte (0x02)
    char header = 0;
    if (m_uart.readBuffer(&header,  len: 1) != 1) return false;
    /* Validate STX - if not 0x02, we have garbage data.
       Clean buffer to avoid synchronization issues. */
    if (header != RDM_STX) {
        char trash[32];
        if (waitForData( timeout_ms: 10)) {    // Brief wait for remaining garbage
            while(m_uart.readBuffer(trash,  len: sizeof(trash)) > 0);
        }
        return false;
    }
    // Start building frame: STX at position 0
    m_rawBuffer[0] = RDM_STX;
    int bytesToRead = 13;    // Remaining: 10 data + 2 checksum + 1 ETX
    int totalRead = 0;
    /* Read frame body with 100ms timeout.
       At 9600 bps (~1ms/byte), 100ms is ample for 13 bytes. */
    while (totalRead < bytesToRead) {
        if (!waitForData( timeout_ms: 100)) {
            return false;    // Frame incomplete/corrupted
        }

        int n = m_uart.readBuffer(m_rawBuffer + 1 + totalRead,
                                  len: bytesToRead - totalRead);
        if (n > 0) {
            totalRead += n;
        } else {
            return false;    // Read error
        }
    }
```

verifies if data was received on UART

Header Validation: Discards random noise to ensure a clean read.

Payload Retrieval: Captures the remaining frame bytes with strict timeout control.

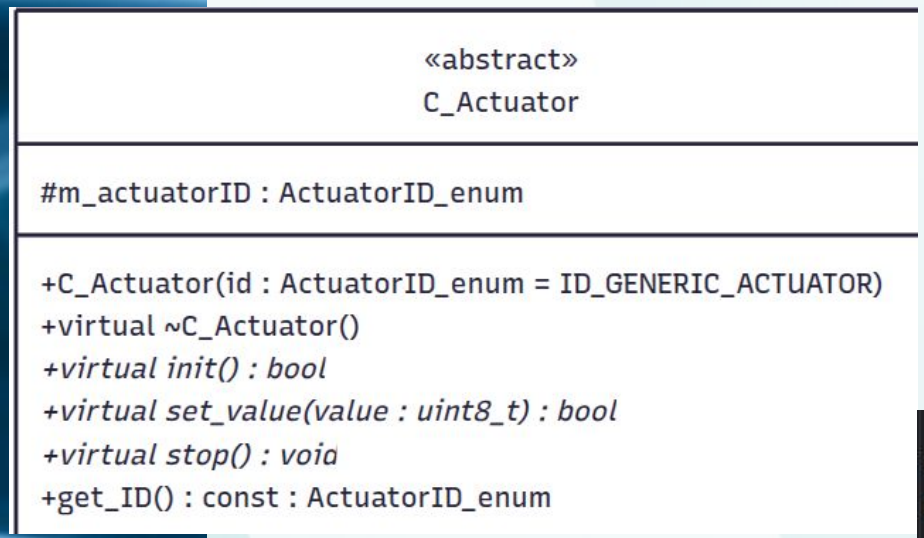
CENTROALGORITMI

```
    // Validate frame structure and checksum
    bool success = false;
    if (m_rawBuffer[13] == RDM_ETX) {
        if (validateChecksum(m_rawBuffer)) {
            success = true;
            if (data) {
                data->type = ID_RDM6300;
                parseTag(m_rawBuffer, data->data.rfid_single.tagID);
            }
        }
    }
    /* Non-blocking buffer cleanup.
       Remove any leftover bytes to keep UART sync. */
    if (waitForData( timeout_ms: 0)) {
        char trash[64];
        while(m_uart.readBuffer(trash,  len: sizeof(trash)) > 0);
    }
    return success;
}
```

Frame Verification: Validates the End-Byte (ETX) and Checksum before extracting the ID.

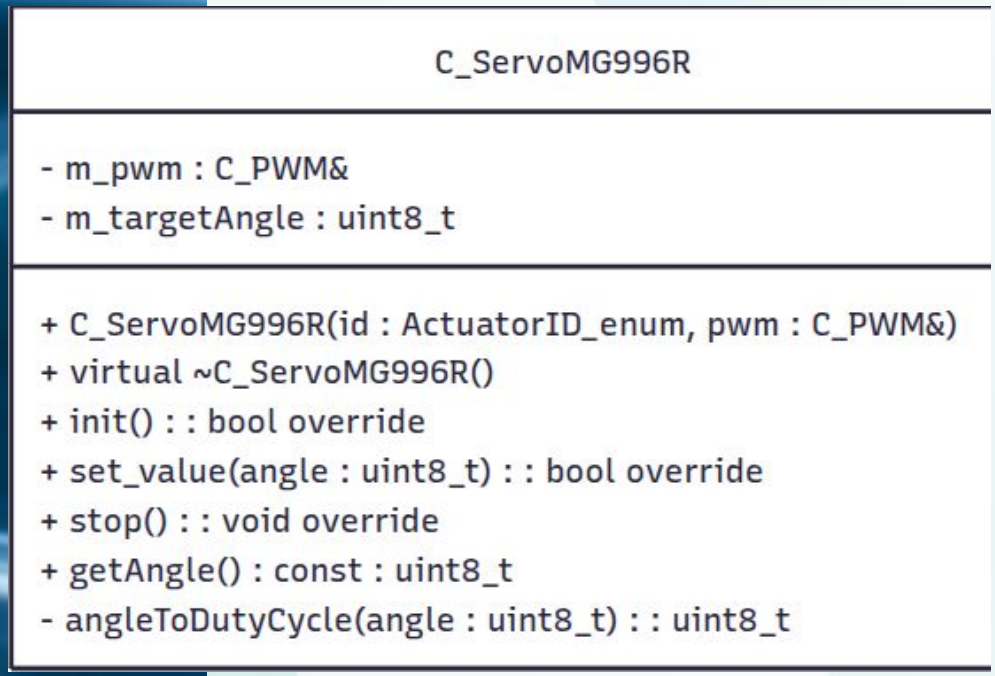
Post-Read Flush: Clears any residual data to prevent overlap in the next cycle.

C_Actuator




```
class C_Actuator {
protected:
    ActuatorID_enum m_actuatorID;

public:
    C_Actuator(ActuatorID_enum id): m_actuatorID(id) {}
    virtual ~C_Actuator();
    virtual bool init() = 0;
    virtual bool set_value(uint8_t value) = 0;
    virtual void stop() = 0;

    ActuatorID_enum get_ID() const { return m_actuatorID; }
};

#endif // C_ACTUATOR_H
```

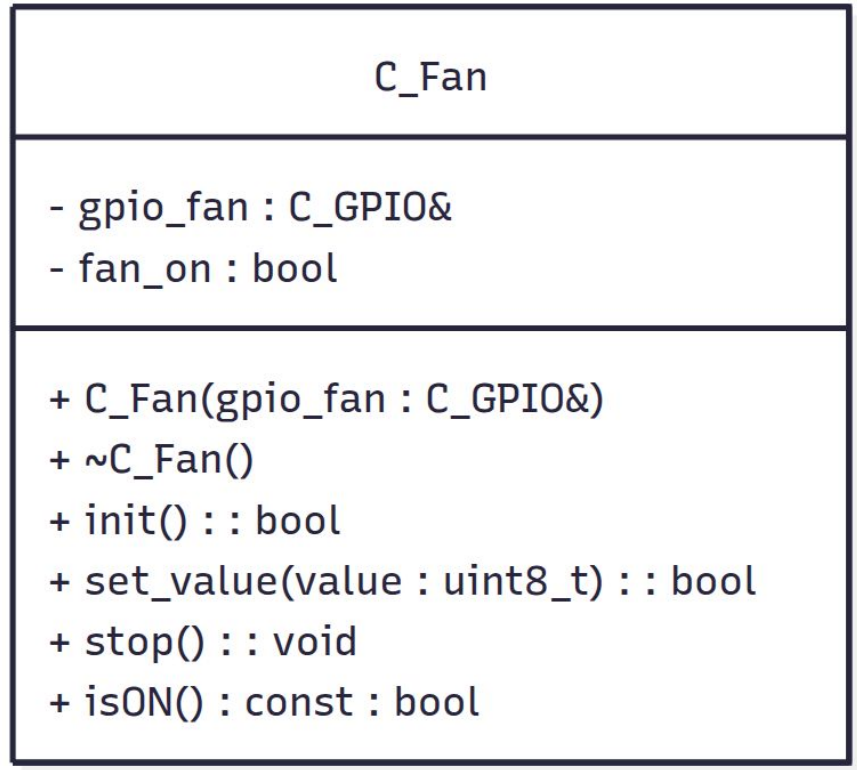
Class C_Servo




```cpp
#ifndef C_SERVOMG996R_H
#define C_SERVOMG996R_H

#include "C_Actuator.h"
#include "../hal/C_PWM.h"
#include <cstdint>

class C_ServoMG996R : public C_Actuator {
public:
    C_ServoMG996R(ActuatorID_enum id, C_PWM& pwm);

    ~C_ServoMG996R() override;

    bool init() override;
    bool set_value(uint8_t angle) override;
    void stop() override;

    uint8_t getAngle() const { return m_targetAngle; }

private:
    C_PWM& m_pwm;
    uint8_t m_targetAngle;

    static uint8_t angleToDutyCycle(uint8_t angle);

};
#endif // C_SERVOMG996R_H
```

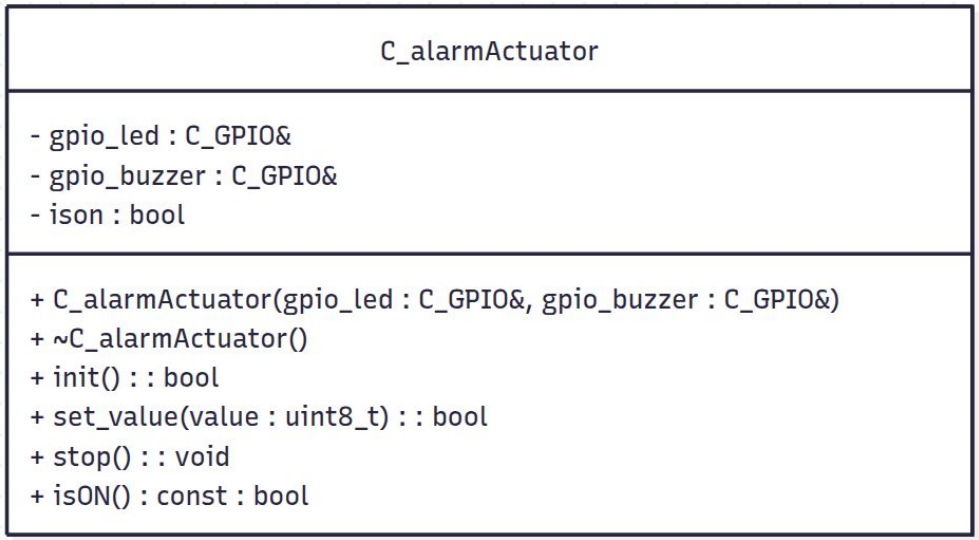
Class C_Fan


```cpp
#ifndef C_FAN_H
#define C_FAN_H

#include "C_Actuator.h"

class C_GPIO;

class C_Fan final: public C_Actuator {

public:
    C_Fan(C_GPIO& gpio_fan);
    ~C_Fan() override;
    bool init() override;
    bool set_value(uint8_t value) override;
    void stop() override;
    bool isON() const {return fan_on;}
private:
    C_GPIO& gpio_fan;
    bool fan_on;
};

#endif #ifndef C_FAN_H
```



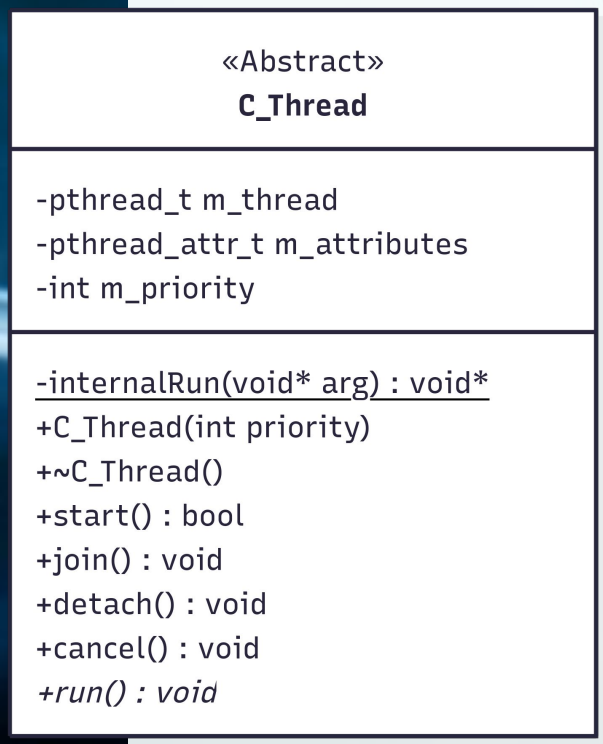
Class C_Alarm


```cpp
#ifndef C_ALARMACTUATOR_H
#define C_ALARMACTUATOR_H
#include "C_Actuator.h"
#include "SharedTypes.h"

class C_GPIO;

class C_alarmActuator final : public C_Actuator {

public:
    C_alarmActuator(C_GPIO& gpio_led, C_GPIO& gpio_buzzer);
    ~C_alarmActuator() override;
    bool init() override;
    bool set_value(uint8_t value) override;
    void stop() override;
    bool isON() const {return ison; }
private:
    C_GPIO& gpio_led;
    C_GPIO& gpio_buzzer;
    bool ison;
};

#endif #ifndef C_ALARMACTUATOR_H
```



Class C_thread

- All of the other thread classes inherit from the C_Thread base class.
- The C_Thread class enables derived classes to avoid pthread library handling




```cpp
class C_Thread {

    // --- Private Data ---
    pthread_t m_thread;           // Stores the Thread ID.
    pthread_attr_t m_attributes;  // Stores settings like priority.
    int m_priority;               // Real-Time priority value.

    // Bridge function required to connect C++ objects to C-style pthreads.
    static void* internalRun(void* arg);

public:

    // Constructor : fill the attributes(priority,...)
    C_Thread(int priority = 0);

    // Destructor: Cleans up the attributes.
    virtual ~C_Thread();

    // --- Execution ---

    // Calls 'pthread_create'(threadid, attributs and run function)
    bool start();

    // Wrapper: Calls 'pthread_join'
    void join();

    // Wrapper: Calls 'pthread_detach'
    void detach();

    // Wrapper: Calls 'pthread_cancel'
    void cancel();


    // Pure Virtual Function. Derived classes must implement their own run!
    virtual void run() = 0;
};
```

```
bool C_Thread::start() {

    int result = pthread_create(&m_thread, &m_attributes, internalRun, arg: this);
```

Pointer to the object that called start function

```
function
extern int pthread_create(
    pthread_t *__newthread,
    const pthread_attr_t *__attr,
    void *(*__start_routine)(void *),
    void *__arg) noexcept(true)
```

We want to use pthread_create for a specif object and with this start the specific run function of that object

Can't do it directly because pthread_create is not expecting a c++ type method that has an implicit "this" pointer to the specific object

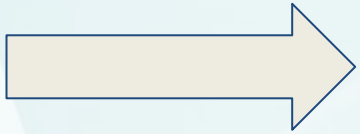
```
static void* internalRun(void* arg);
```

```
void* C_Thread::internalRun(void* arg) {
    C_Thread* threadObj = (C_Thread*)arg;//casting back to C_thread object pointer

    if (threadObj != NULL) {
        threadObj->run();
    }

    return NULL;
}
```

Class C_Monitor

Condition Variable + Mutex

C_Monitor
-pthread_mutex_t m_mutex -pthread_cond_t m_cond
+C_Monitor() +~C_Monitor() +wait() +signal()

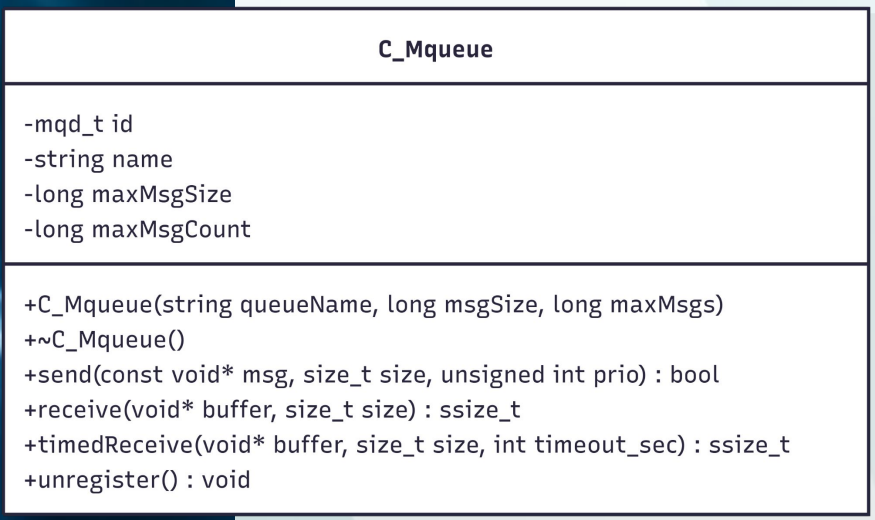


```
class C_Monitor {
    pthread_mutex_t m_mutex;
    pthread_cond_t m_cond;

public:
    C_Monitor();
    ~C_Monitor();

    void wait();
    void signal();
};
```

CENTROALGORITMI

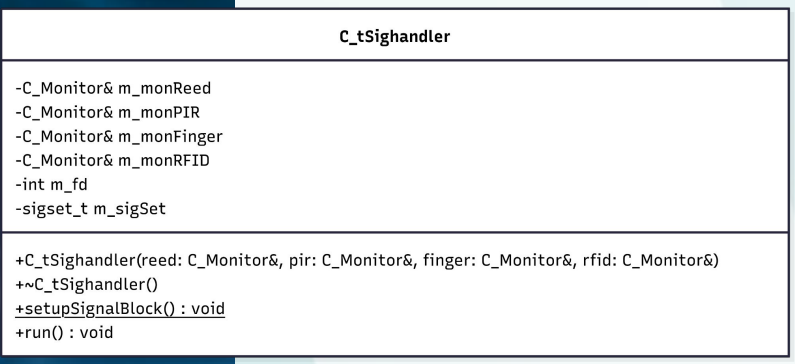
```
C_Monitor::C_Monitor() {
    if (pthread_mutex_init(&m_mutex, mutexattr: NULL) != 0){
        cerr << "Mutex init failed" << endl;
    }
    if (pthread_cond_init(&m_cond, cond_attr: NULL) != 0) {
        cerr << "Cond init failed" << endl;
    }
}

C_Monitor::~C_Monitor() {
    pthread_mutex_destroy(&m_mutex);
    pthread_cond_destroy(&m_cond);
}

void C_Monitor::wait() {

    pthread_mutex_lock(&m_mutex);
    pthread_cond_wait(&m_cond, &m_mutex);
    pthread_mutex_unlock(&m_mutex);
}


void C_Monitor::signal() {
    pthread_mutex_lock(&m_mutex);
    pthread_cond_signal(&m_cond);
    pthread_mutex_unlock(&m_mutex);
```

Class C_mqueue




```
class C_Mqueue {
private:
    mqd_t id;
    string name;
    long maxMsgSize;//message nax byte syze
    long maxMsgCount;//max message number inside queue

public:
    C_Mqueue(string queueName, long msgSize = 1024, long maxMsgs = 10);

    ~C_Mqueue();


    bool send(const void* msg, size_t size, unsigned int prio = 0);
    //thread waits for message queue
    ssize_t receive(void* buffer, size_t size);
    //thread waits for a specific time and if there is no message just keeps going
    ssize_t timedReceive(void* buffer, size_t size, int timeout_sec);
    void unregister();
};

#endif // C_MQUEUE_H
```

Class C_tSighandler




```
class C_tSighandler : public C_Thread {

    C_Monitor& m_monReed;
    C_Monitor& m_monPIR;
    C_Monitor& m_monFinger;
    C_Monitor& m_monRFID;

    int m_fd;
    sigset_t m_sigSet;

public:
    C_tSighandler(C_Monitor& reed, C_Monitor& pir, C_Monitor& finger, C_Monitor& rfid);

    virtual ~C_tSighandler() override;

    // should be the first thing to be called in the main
    static void setupSignalBlock();


    void run() override;
};
```

ESRG

Embedded Systems Research Group

CENTROALGORITMI

```
void C_tSighandler::setupSignalBlock() {

    sigset_t set;
    sigemptyset(&set);
    sigaddset(&set, signo:43);
    sigaddset(&set, signo:44);
    sigaddset(&set, signo:45);
    sigaddset(&set, signo:46);
    sigaddset(&set, signo:47);
    sigaddset(&set, signo:48);

    if (pthread_sigmask( how: SIG_BLOCK, &set, oldmask: NULL) != 0) {
        perror( s: "Erro ao bloquear sinais");
    }
}

void C_tSighandler::run() {
    siginfo_t info;


    m_fd = open( file: "/dev/irq0", oflag: O_WRONLY);
    if (m_fd < 0 || ioctl(m_fd, request: REGIST_PID, 0) < 0) {
        std::cerr << "[Sighandler] ERRO: Não foi possivel conectar a
        return;
    }

    std::cout << "[Sighandler] Pronto. À espera de eventos de hardwa
```

```
int sig = sigtimedwait(&m_sigSet, &info, &timeout);

if (sig < 0) {
    if (errno == EAGAIN) {
        continue;
    }
    continue;
}

int pino = info.si_int;

switch (sig) {
    case 43:
        std::cout << "[Hardware] Reed Switch detetado
        m_monReed_vault.signal();
        break;

    case 44:
        std::cout << "[Hardware] Reed Switch detetado
        m_monReed_room.signal();
        break;
    case 45:
        std::cout << "[Hardware] Movimento PIR detetad
        m_monPIR.signal();
        break;
    case 46:
        std::cout << "[Hardware] Digital lida no pino
        m_monFinger.signal();
        break;
    case 47:
        std::cout << "[Hardware] RFID entrada no pino
        m_monRFID_entry.signal();
        break;
    case 48:
        std::cout << "[Hardware] RFID saida aproximad
        m_monRFID_exit.signal();
}
```

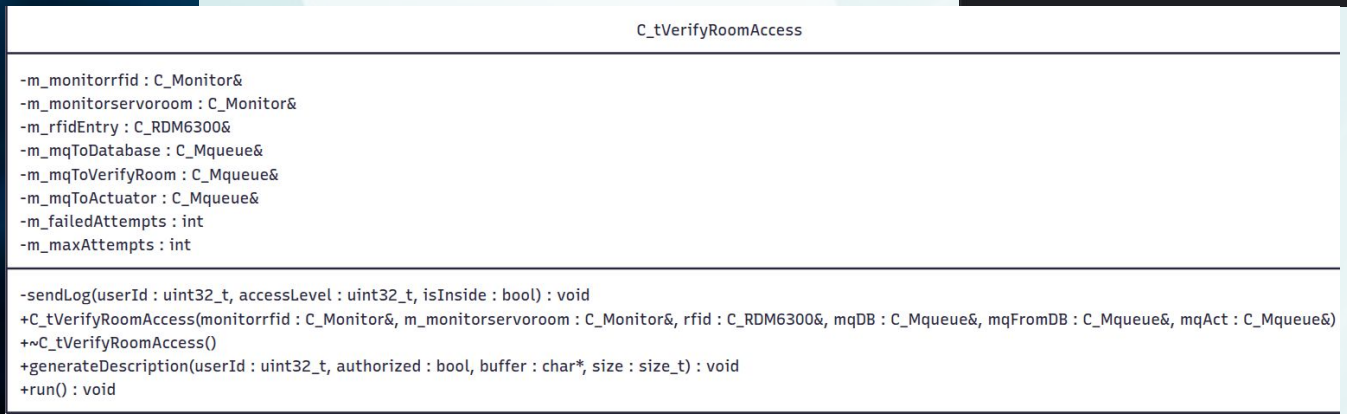
ESRG

Embedded Systems Research Group

C_tVerifyRoomAccess

CENTROALGORITMI

```
class C_tVerifyRoomAccess : public C_Thread {
private:

    C_Monitor& m_monitorrfid;
    C_Monitor& m_monitorservoroom;
    C_RDM6300& m_rfidEntry;
    C_Mqueue& m_mqToDatabase;
    C_Mqueue& m_mqToVerifyRoom;
    C_Mqueue& m_mqToActuator;

    int m_failedAttempts;
    int m_maxAttempts;

    void sendLog(uint32_t userId, uint32_t accessLevel, bool isInside);

public:

    _tVerifyRoomAccess(C_Monitor& monitorrfid,
                       C_Monitor& m_monitorservoroom,
                       C_Mqueue& mqDB,
                       C_Mqueue& mqFromDB,
                       C_Mqueue& mqAct);

    virtual ~C_tVerifyRoomAccess();
    void generateDescription(uint32_t userId, bool authorized, char* buffer, size_t size);
    void run() override;
};

#endif  #ifndef C_TVERIFYROOMACCESS_H
```



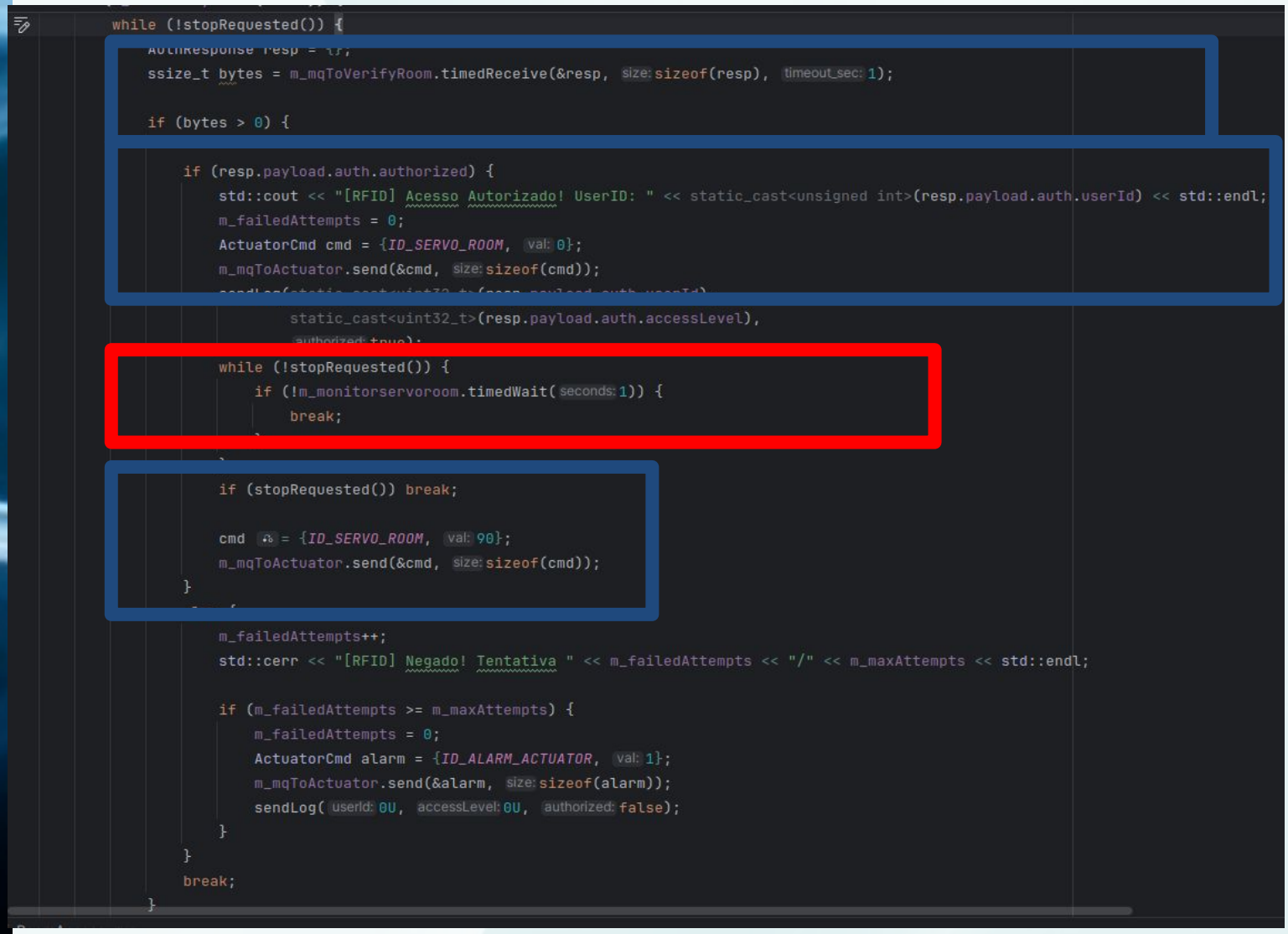
C_tVerifyRoomAccess


```
void C_tVerifyRoomAccess::run() {
    std::cout << "[VerifyRoomAccess] Thread iniciada. À espera de tags..." << std::endl;


    while (!stopRequested()) {

        if (m_monitorrfid.timedWait( seconds: 1)) {
            continue;
        }

        SensorData data = {};
        if (m_rfidEntry.read(&data)) {
            const char* rfidRead = data.data.rfid_single.tagID;
            std::cout << "[RFID entry] Cartão lido: " << rfidRead << std::endl;

            DatabaseMsg msg = {};
            msg.command = DB_CMD_ENTER_ROOM_RFID;
            strncpy(msg.payload.rfid, rfidRead, n: sizeof(msg.payload.rfid) - 1);
            msg.payload.rfid[sizeof(msg.payload.rfid) - 1] = '\0';
            m_mqToDatabase.send(&msg, size: sizeof(msg));
            std::cout << "m enviafa: " << std::endl;
```

ESRG

Embedded Systems Research Group

CENTROALGORITMI



Class dDatabase


```
class dDatabase {
public:
    dDatabase(const std::string& dbPath,
              C_Mqueue& mqDb,
              C_Mqueue& mqRfidIn,
              C_Mqueue& mqRfidOut,
              C_Mqueue& mqFinger,
              C_Mqueue& mqCheckMovement,
              C_Mqueue& mqToWeb,
              C_Mqueue& mqToEnv);
    ~dDatabase();

    bool open();
    void close();
    bool initializeSchema();
    void processDbMessage(const DatabaseMsg& msg);

private:
    sqlite3* m_db;
    std::string m_dbPath;

    C_Mqueue& m_mqToDatabase;
    C_Mqueue& m_mqToVerifyRoom;
    C_Mqueue& m_mqToLeaveRoom;
    C_Mqueue& m_mqToFingerprint;
    C_Mqueue& m_mqToCheckMovement;
    C_Mqueue& m_mqToWeb;
    C_Mqueue& m_mqToEnvThread;
```

```
void handleAccessRequest(const char* rfid, bool isEntering);
void handleScanInventory(const Data_RFID_Inventory& inventory);
void handleCheckUserInPir();
void handleLogin(const LoginRequest& login);
void handleGetDashboard();
void handleGetSensors();
void handleGetActuators();
void handleInsertLog(const DatabaseLog& log);
void updateSensorTable(uint8_t entityID, double value, double value2, uint32_t timestamp)
void updateActuatorTable(uint8_t entityID, double value, uint32_t timestamp);

void handleRegisterUser(const UserData& user);
void handleGetUsers();
void handleCreateUser(const UserData& user);
void handleModifyUser(const UserData& user);
void handleRemoveUser(uint32_t userId);

void handleGetAssets();
void handleCreateAsset(const AssetData& asset);
void handleModifyAsset(const AssetData& asset);
void handleRemoveAsset(const char* tag);

void handleGetSettings();
void handleGetSettingsForThread();

void handleUpdateSettings(const SystemSettings& settings);

void handleFilterLogs(const LogFilter& filter);
```

database open()


```
}

bool dDatabase::open() {
    int result = sqlite3_open( filename: m_dbPath.c_str(), &m_db);

    if (result != SQLITE_OK) {
        std::cerr << "ERRO: Não foi possível abrir a base de dados: "
                  << sqlite3_errmsg(m_db) << std::endl;
        return false;
    }

    std::array<unsigned char, kDbKeyLength> key = buildDbKey();

    if (sqlite3_key(m_db,  pKey: key.data(), static_cast<int>(key.size())) != SQLITE_OK) {
        std::cerr << "ERRO: Falha ao definir chave SQLCipher: "
                  << sqlite3_errmsg(m_db) << std::endl;
        secureZero(key.data(),  len: key.size());
        sqlite3_close(m_db);
        m_db = nullptr;
        return false;
    }

    secureZero(key.data(),  len: key.size());

    std::cout << "SUCESSO: Ligado ao ficheiro: " << m_dbPath << std::endl;
    return true;
}
```

```
constexpr std::array<uint8_t, kDbKeyLength> kDbKeyPart = {
    0x31, 0x76, 0x9a, 0x40, 0x7b, 0xa3, 0x1d, 0xc4,
    0x2a, 0x8f, 0x61, 0x05, 0xde, 0x19, 0x73, 0xb0,
    0x4c, 0x8a, 0x16, 0xf2, 0x9c, 0x5b, 0x07, 0xd8,
    0x21, 0x90, 0x4e, 0xbc, 0x63, 0x0a, 0xe1, 0x57
};

constexpr std::array<uint8_t, kDbKeyLength> kDbKeyMask = {
    0x8d, 0x1f, 0x27, 0xbb, 0x19, 0x5e, 0x88, 0x22,
    0x9c, 0x30, 0xf5, 0x6a, 0x4b, 0xac, 0x02, 0x1e,
    0x77, 0x0d, 0xf9, 0x45, 0x6b, 0x92, 0xfe, 0x14,
    0xc8, 0x59, 0x73, 0x02, 0xbd, 0xa4, 0x38, 0xc1
};

std::array<unsigned char, kDbKeyLength> buildDbKey() {
    std::array<unsigned char, kDbKeyLength> key{};
    for (size_t i = 0; i < key.size(); ++i) {
        key[i] = static_cast<unsigned char>(kDbKeyPart[i] ^ kDbKeyMask[i]);
    }
    return key;
}

void secureZero(void* data, size_t len) {
    volatile unsigned char* p = static_cast<volatile unsigned char*>(data);
    while (len--) {
        *p++ = 0;
    }
}
```

ESRG
Embedded Systems Research Group

database initializeSchema()

CENTROALGORITMI

Database Tables
Users
Logs
Assets
Sensors
Actuators
SystemSettings

sqlite> PRAGMA table_info(Users);

cid	name	type	notnull	dflt_value	pk
0	UserID	INTEGER	0		1
1	Name	TEXT	0		0
2	RFID_Card	TEXT	0		0
3	FingerprintID	INTEGER	0		0
4	Password	TEXT	0		0
5	AccessLevel	INTEGER	0		0
6	IsInside	INTEGER	0	0	0

sqlite> PRAGMA table_info(Logs);

cid	name	type	notnull	dflt_value	pk
0	LogsID	INTEGER	0		1
1	EntityID	INTEGER	0		0
2	Timestamp	INTEGER	0		0
3	LogType	INTEGER	0		0
4	Description	TEXT	0		0
5	Value	INTEGER	0		0
6	Value2	INTEGER	0	0	0

sqlite> PRAGMA table_info(Assets);

cid	name	type	notnull	dflt_value	pk
0	AssetID	INTEGER	0		1
1	Name	TEXT	0		0
2	RFID_Tag	TEXT	0		0
3	State	TEXT	0		0
4	LastRead	INTEGER	0		0

sqlite> PRAGMA table_info(Sensors);

cid	name	type	notnull	dflt_value	pk
0	SensorID	INTEGER	0		1
1	Type	TEXT	0		0
2	Value	INTEGER	0		0
3	LastUpdate	INTEGER	0	0	0

sqlite> PRAGMA table_info(Actuators);

cid	name	type	notnull	dflt_value	pk
0	ActuatorID	INTEGER	0		1
1	Type	TEXT	0		0
2	State	INTEGER	0		0
3	LastUpdate	INTEGER	0	0	0

sqlite> PRAGMA table_info(SystemSettings);

cid	name	type	notnull	dflt_value	pk
0	ID	INTEGER	0		1
1	TempThreshold	INTEGER	0	30	0
2	SamplingTime	INTEGER	0	600	0

Database main


```
while (!g_stop) {
    DatabaseMsg msg{};
    ssize_t bytesRead = mqToDb.timedReceive(&msg, size: sizeof(DatabaseMsg), timeout_sec: 1);
    if (bytesRead > 0) {
        db.processDbMessage(msg);
    }
}
```

ESRG

Embedded Systems Research Group

CENTROALGORITMI

```
void dDatabase::processDbMessage(const DatabaseMsg &msg) {
        case DB_CMD_ENTER_ROOM_RFID:
            handleAccessRequest(msg.payload.rfid, isEntering: true);
        case DB_CMD_LEAVE_ROOM_RFID:
            handleAccessRequest(msg.payload.rfid, isEntering: false);
            break;
        case DB_CMD_UPDATE_ASSET:
            handleScanInventory(msg.payload.rfidInventory);
            break;
        case DB_CMD_WRITE_LOG:
            handleInsertLog(msg.payload.log);
            break;
        case DB_CMD_USER_IN_PIR:
            handleCheckUserInPir();
            break;
        case DB_CMD_LOGIN:
            handleLogin(msg.payload.login);
            break;
        case DB_CMD_GET_DASHBOARD:
            handleGetDashboard();
            break;
        case DB_CMD_GET_SENSORS:
            handleGetSensors();
            break;
        case DB_CMD_GET_ACTUATORS:
            handleGetActuators();
            break;
        case DB_CMD_REGISTER_USER:
            handleRegisterUser(msg.payload.user);
            break;
        case DB_CMD_GET_USERS:
            handleGetUsers();
            break;
        case DB_CMD_CREATE_USER:
            handleCreateUser(msg.payload.user);
```

ESRG

Embedded Syst

Research Group

DALGORITMI

```
void dDatabase::handleAccessRequest(const char* rfid, bool isEntering) {
    sqlite3_stmt* stmt;

    AuthResponse resp = {};
    resp.command = isEntering ? DB_CMD_ENTER_ROOM_RFID : DB_CMD_LEAVE_ROOM_RFID;

    const char* sqlSelect = "SELECT UserID, AccessLevel FROM Users WHERE RFID_Card = ?;";
    if (sqlite3_prepare_v2(m_db, sqlSelect, nByte: -1, &stmt, pzTail: nullptr) == SQLITE_OK) {
        sqlite3_bind_text(stmt, 1, rfid, -1, SQLITE_TRANSIENT);
        if (sqlite3_step(stmt) == SQLITE_ROW) {
            resp.payload.auth.authorized = true;
            resp.payload.auth.userId = static_cast<uint32_t>(sqlite3_column_int(stmt, iCol: 0));
            resp.payload.auth.accessLevel = static_cast<uint32_t>(sqlite3_column_int(stmt, iCol: 1));

            cout << "\n[CHECK] User Encontrado!" << endl;
            cout << " > UserID:      " << resp.payload.auth.userId << endl;
            cout << " > AccessLevel: " << resp.payload.auth.accessLevel << endl;
            cout << "----------------------------" << endl;
        }
        sqlite3_finalize(stmt);
    }
    if (resp.payload.auth.authorized) {
        int newState = isEntering ? 1 : 0;

        const char* sqlUpdate = "UPDATE Users SET IsInside = ? WHERE UserID = ?;";
        if (sqlite3_prepare_v2(m_db, sqlUpdate, nByte: -1, &stmt, pzTail: nullptr) == SQLITE_OK) {
            sqlite3_bind_int(stmt, 1, newState);
            sqlite3_bind_int(stmt, 2, static_cast<int>(resp.payload.auth.userId));
            sqlite3_step(stmt);
            sqlite3_finalize(stmt);
        }
    }

    if (isEntering) {
        m_mqToVerifyRoom.send(&resp, size: sizeof(resp));
    } else {
        m_mqToLeaveRoom.send(&resp, size: sizeof(resp));
    }
}
```

Class Webserver


```cpp
#ifndef DWEBSERVER_H
#define DWEBSERVER_H

#include <string>
#include <map>
#include <mutex>
#include "mongoose.h"
#include "nlohmann/json.hpp"
#include "C_Mqueue.h"
#include "SharedTypes.h"

struct SessionData {
    uint32_t userId;
    std::string username;
    uint32_t accessLevel;  // 0=Viewer, 1=Room, 2=Room+Vault
    time_t expires;
};

class dWebServer {
private:
    struct mg_mgr m_mgr;
    C_Mqueue& m_mqToDatabase;
    C_Mqueue& m_mqFromDatabase;
    std::map<std::string, SessionData> m_sessions;
    std::mutex m_sessionMutex;
    int m_port;
    bool m_running;

public:
    dWebServer(C_Mqueue& toDb, C_Mqueue& fromDb, int port = 8080);
    ~dWebServer();

    bool start();
    void stop();
    void run();
```

```cpp
private:
    static void eventHandler(struct mg_connection* c, int ev, void* ev_data);

    void handleLogin(struct mg_connection* c, struct mg_http_message* hm);
    void handleRegister(struct mg_connection* c, struct mg_http_message* hm);
    void handleLogout(struct mg_connection* c, struct mg_http_message* hm);

    void handleDashboard(struct mg_connection* c, struct mg_http_message* hm);
    void handleSensors(struct mg_connection* c, struct mg_http_message* hm);
    void handleActuators(struct mg_connection* c, struct mg_http_message* hm);
    void handleLogsFilter(struct mg_connection* c, struct mg_http_message* hm);

    void handleUsers(struct mg_connection* c, struct mg_http_message* hm);
    void handleUsersById(struct mg_connection* c, struct mg_http_message* hm);
    void handleAssets(struct mg_connection* c, struct mg_http_message* hm);
    void handleAssetsById(struct mg_connection* c, struct mg_http_message* hm);
    void handleSettings(struct mg_connection* c, struct mg_http_message* hm);

    std::string generateToken();
    bool validateSession(struct mg_http_message* hm, SessionData& outSession);
    void cleanExpiredSessions();

    void sendJson(struct mg_connection* c, int statusCode, const nlohmann::json& data);
    void sendError(struct mg_connection* c, int statusCode, const std::string& message);

    static bool matchUri(const struct mg_str* uri, const char* pattern);
    static bool matchPrefix(const struct mg_str* uri, const char* pattern);
};

#endif #ifndef DWEBSERVER_H
```

CENTROALGORITMI

```cpp
bool dWebServer::start() {
    std::string addr = "http://10.42.0.163:" + std::to_string(m_port) + "/";

    if (mg_http_listen(&m_mgr, url: addr.c_str(), eventHandler, fn_data: this) == nullptr) {
        std::cerr << "[WebServer] Failed to start on port " << m_port << std::endl;
        return false;
    }

    m_running = true;
    std::cout << "[WebServer] Listening on " << addr << std::endl;
    return true;
}
```

Service Initialization: Binding the HTTP listener to the network interface and attaching the global event dispatcher.

```cpp
void dWebServer::stop() {
    m_running = false;
}
```

```cpp
void dWebServer::run() {
    while (m_running) {
        mg_mgr_poll(&m_mgr, ms: 1000);
        cleanExpiredSessions();
    }
}
```

Main Execution Loop: Orchestrating non-blocking event processing and background session maintenance.

CENTROALGORITMI

```
void dWebServer::eventHandler(struct mg_connection* c, int ev, void* ev_data) {
    if (ev == MG_EV_HTTP_MSG) {
        struct mg_http_message* hm = (struct mg_http_message*)ev_data;
        dWebServer* self = (dWebServer*)c->fn_data;

        if (matchUri(&hm->uri, pattern: "/api/login")) {
            self->handleLogin(c, hm);
        }
        else if (matchUri(&hm->uri, pattern: "/api/register")) {
            self->handleRegister(c, hm);
        }
        else if (matchUri(&hm->uri, pattern: "/api/logout")) {
            self->handleLogout(c, hm);
        }
        else if (matchUri(&hm->uri, pattern: "/api/dashboard")) {
            self->handleDashboard(c, hm);
        }
        else if (matchUri(&hm->uri, pattern: "/api/sensors")) {
            self->handleSensors(c, hm);
        }
        else if (matchUri(&hm->uri, pattern: "/api/actuators")) {
            self->handleActuators(c, hm);
        }
        else if (matchUri(&hm->uri, pattern: "/api/logs/filter")) {
            self->handleLogsFilter(c, hm);
        }
        else if (matchPrefix(&hm->uri, pattern: "/api/assets/")) {
            self->handleAssetsById(c, hm);
        }
        else if (matchPrefix(&hm->uri, pattern: "/api/users/")) {
            self->handleUsersById(c, hm);
        }
        else if (matchUri(&hm->uri, pattern: "/api/users")) {
            self->handleUsers(c, hm);
        }
        else if (matchUri(&hm->uri, pattern: "/api/assets")) {
            self->handleAssets(c, hm);
        }
```

```
        else if (matchUri(&hm->uri, pattern: "/api/settings")) {
            self->handleSettings(c, hm);
        }
        else if (matchUri(&hm->uri, pattern: "/")) {
            mg_http_reply(c, status_code: 302, headers: "Location: /login.html\r\n", body_fmt: "");
        }
        else {
            SessionData session;
            bool logado = self->validateSession(hm, [&] session);
            if (matchUri(&hm->uri, pattern: "/users.html") ||
                matchUri(&hm->uri, pattern: "/settings.html") ||
                matchUri(&hm->uri, pattern: "/assets.html")) {
                if (!logado || session.accessLevel < 1) {
                    mg_http_reply(c, status_code: 302, headers: "Location: /login.html\r\n", body_fmt: "");
                    return;
                }
            }
            struct mg_http_serve_opts opts;
            memset(&opts, c: 0, n: sizeof(opts));
            opts.root_dir = "/root/SecureAsset/web";
            mg_http_serve_dir(c, hm, &opts);
        }
    }
}
```

Dispatches incoming HTTP requests to specialized backend logic based on URI matching

Static Content Delivery: Automatically serves HTML, CSS, and JS files from the server's root directory.

CENTROALGORITMI

```
void dWebServer::handleSensors(struct mg_connection* c, struct mg_http_message* hm) {
    SessionData session;
    if (!validateSession(hm, [&]session)) {
        sendError(c, statusCode: 401, message: "Not authenticated");
        return;
    }

    DatabaseMsg msg = {};
    msg.command = DB_CMD_GET_SENSORS;
    m_mqToDatabase.send(&msg, size: sizeof(msg));

    DbWebResponse resp = {};
    ssize_t bytes = m_mqFromDatabase.timedReceive(&resp, size: sizeof(resp), timeout_sec: 2);

    if (bytes > 0 && resp.success) {
        sendJson(c, statusCode: 200, data: nlohmann::json::parse([&]resp.jsonData));
    } else {
        sendError(c, statusCode: 500, message: "Failed to get sensors");
    }
}
```

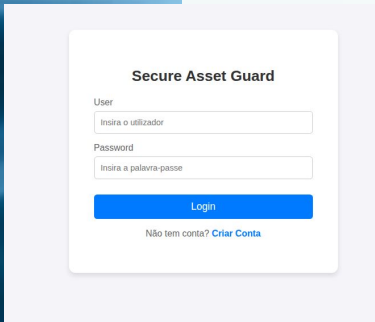
Verifies the user session and fetches current sensor data from the database to send back as JSON.

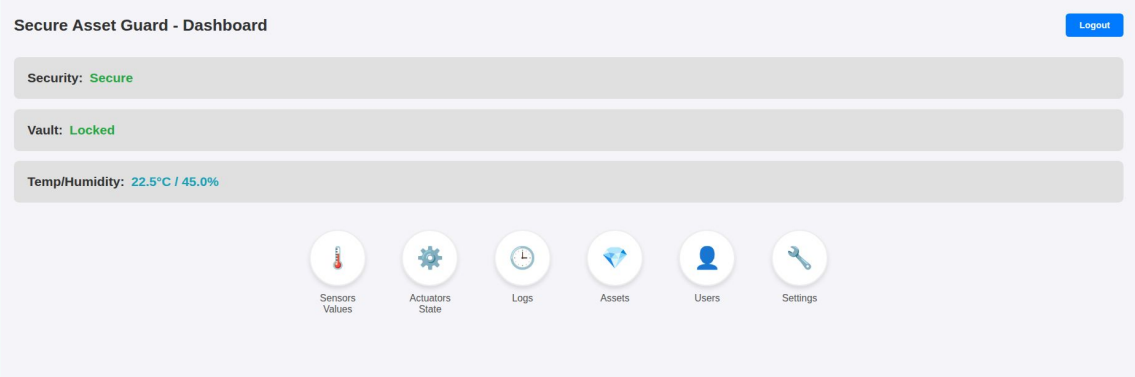
ESRG

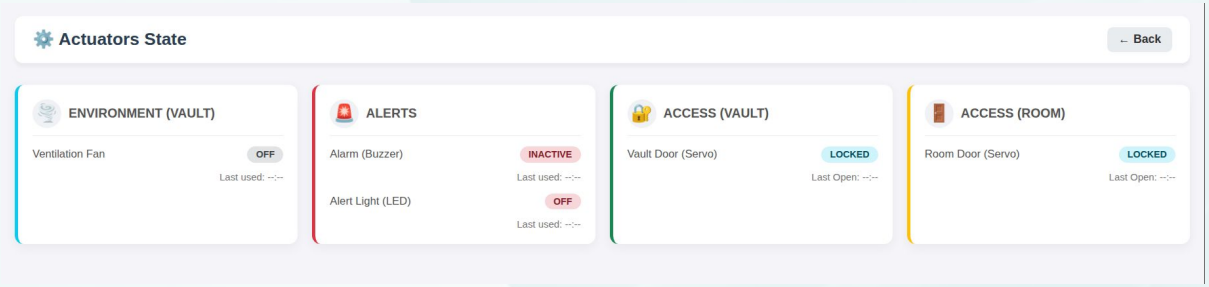
Embedded Systems Research Group

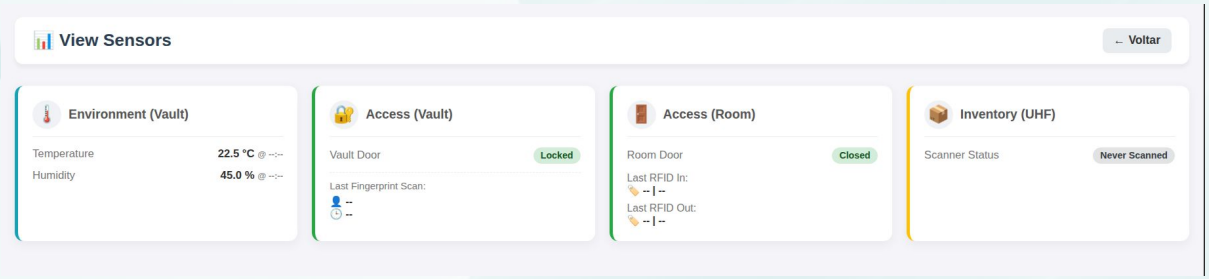
Web Interface

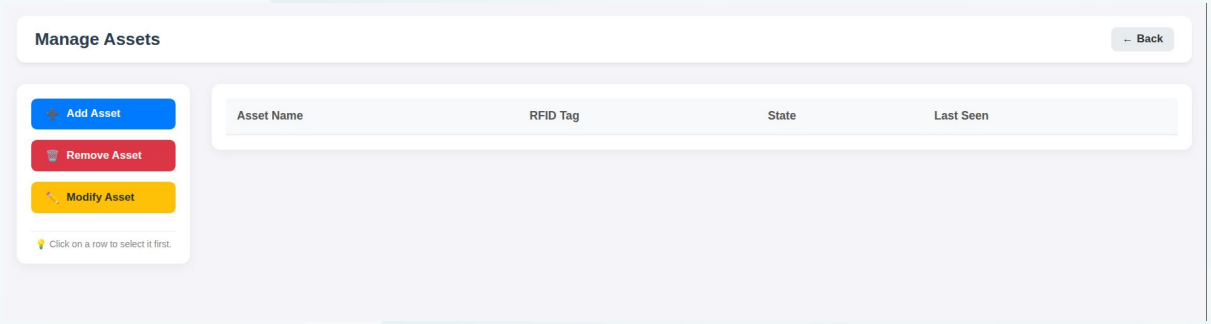
CENTROALGORITMI

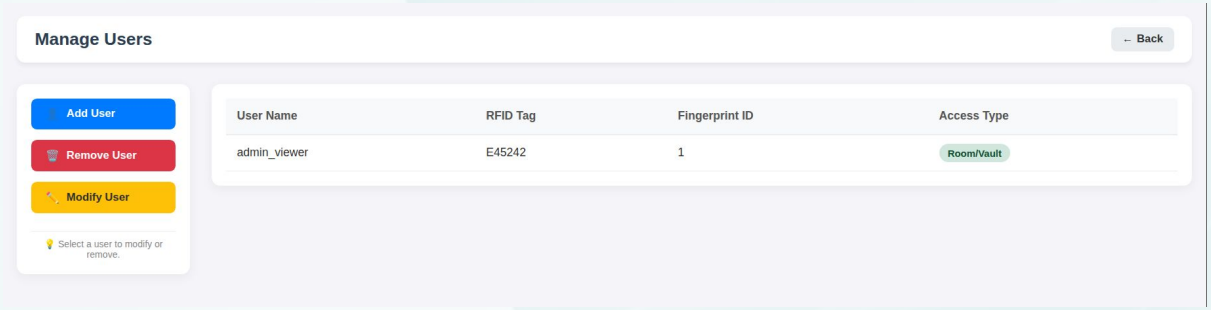


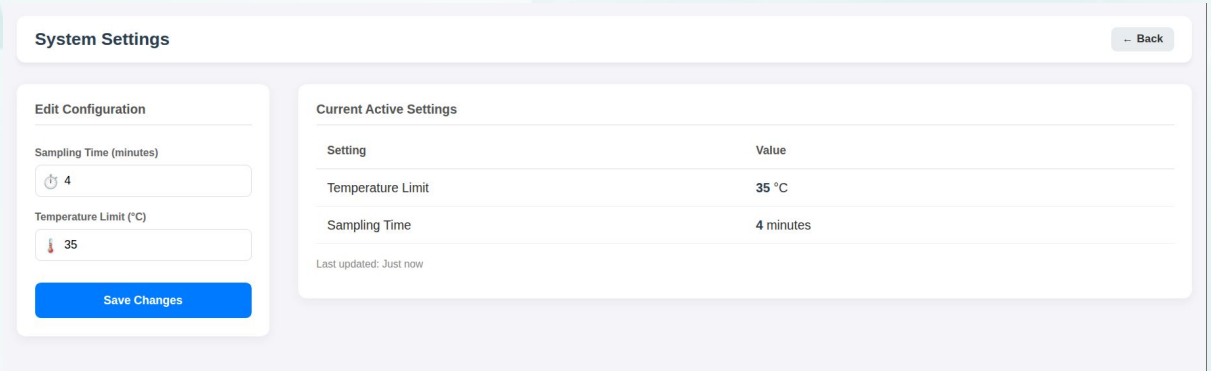




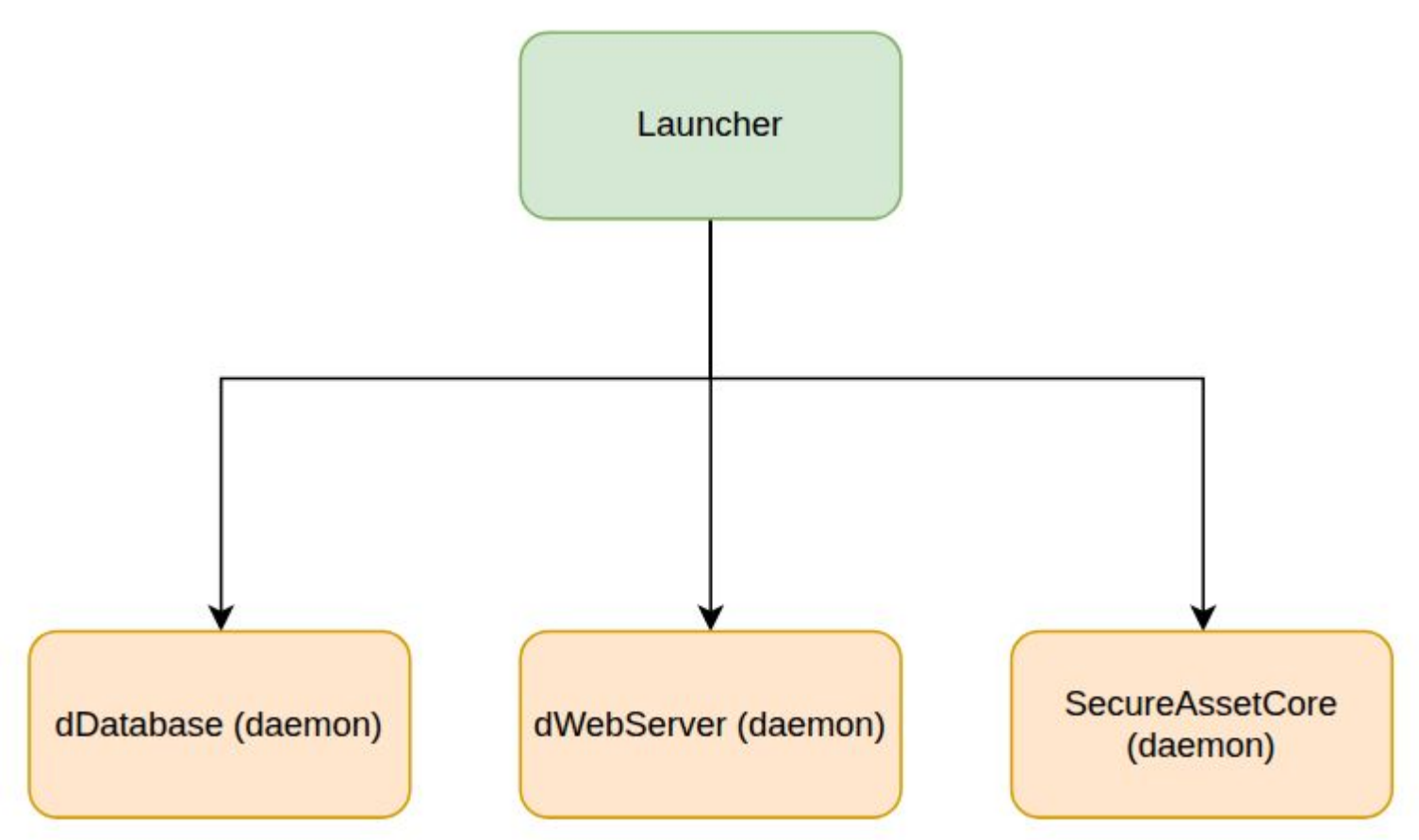



Manage Assets
← Back
Add Asset
Remove Asset
Modify Asset
Click on a row to select it first.
Asset Name
RFID Tag
State
Last Seen


Manage Users
← Back
Add User
Remove User
Modify User
Select a user to modify or remove.
User Name
RFID Tag
Fingerprint ID
Access Type
admin_viewer
E45242
1
Room/Vault


System Settings
← Back
Edit Configuration
Sampling Time (minutes)
4
Temperature Limit (°C)
35
Save Changes
Current Active Settings
Setting
Value
Temperature Limit
35 °C
Sampling Time
4 minutes
Last updated: Just now

Startup & Shutdown Daemonization



ESRG
Embedded Systems
Research Group

Startup & Shutdown Daemonization

CENTROALGORITMI

```
// Create message queues (launcher is the owner)
std::vector<std::unique_ptr<C_Mqueue>> mqs;
try {
    mqs.push_back( x: std::make_unique<C_Mqueue>( queueName: "/mq_to_db",  msgSize: sizeof(DatabaseMsg),  maxMsgs: 20,  createNew: true));
    mqs.push_back( x: std::make_unique<C_Mqueue>( queueName: "/mq_to_actuator",  msgSize: sizeof(ActuatorCmd),  maxMsgs: 20,  createNew: true));
    mqs.push_back( x: std::make_unique<C_Mqueue>( queueName: "/mq_rfid_in",  msgSize: sizeof(AuthResponse),  maxMsgs: 10,  createNew: true));
    mqs.push_back( x: std::make_unique<C_Mqueue>( queueName: "/mq_rfid_out",  msgSize: sizeof(AuthResponse),  maxMsgs: 10,  createNew: true));
    mqs.push_back( x: std::make_unique<C_Mqueue>( queueName: "/mq_move",  msgSize: sizeof(AuthResponse),  maxMsgs: 10,  createNew: true));
    mqs.push_back( x: std::make_unique<C_Mqueue>( queueName: "/mq_finger",  msgSize: sizeof(AuthResponse),  maxMsgs: 10,  createNew: true));
    mqs.push_back( x: std::make_unique<C_Mqueue>( queueName: "/mq_db_to_env",  msgSize: sizeof(AuthResponse),  maxMsgs: 10,  createNew: true));
    mqs.push_back( x: std::make_unique<C_Mqueue>( queueName: "/mq_db_to_web",  msgSize: sizeof(DbWebResponse),  maxMsgs: 10,  createNew: true));
    std::cout << "[Wrapper] Message queues created successfully.\n";
} catch (const std::exception& e) {
    std::cerr << "[Wrapper] ERROR creating queues: " << e.what() << std::endl;
    return EXIT_FAILURE;
}

std::string execDir = getExecutableDir();

DaemonInfo db = launchDaemon( binName:  "dDatabase",  binPath: execDir + "/dDatabase");
if (db.pid < 0) return EXIT_FAILURE;
std::cout << "[Wrapper] dDatabase PID " << db.pid << std::endl;

DaemonInfo web = launchDaemon( binName:  "dWebServer",  binPath: execDir + "/dWebServer");
if (web.pid < 0) {
    stopDaemon( [&] db);
    return EXIT_FAILURE;
}
std::cout << "[Wrapper] dWebServer PID " << web.pid << std::endl;

DaemonInfo core = launchDaemon( binName:  "SecureAssetCore",  binPath: execDir + "/SecureAssetCore");
if (core.pid < 0) {
    stopDaemon( [&] web);
    stopDaemon( [&] db);
    return EXIT_FAILURE;
}
```

Wrapper initializes all IPC message queues and launches each daemon using an explicit READY handshake

ESRG

Embedded Systems Research Group

Startup & Shutdown Daemonization

CENTROALGORITMI

```
static DaemonInfo launchDaemon(const std::string& binName, const std::string& binPath)
    DaemonInfo info;
    info.name = binName;
    int sv_startup[2];
    int sv_shutdown[2];
    if (socketpair( domain: AF_UNIX,  type: SOCK_STREAM,  protocol: 0, sv_startup) < 0) {
        std::perror( s: "socketpair startup");
        return info;
    }
    if (socketpair( domain: AF_UNIX,  type: SOCK_STREAM,  protocol: 0, sv_shutdown) < 0) {
        std::perror( s: "socketpair shutdown");
        close(sv_startup[0]);
        close(sv_startup[1]);
        return info;
    }
```

Create two dedicated IPC channels: STARTUP (READY) and SHUTDOWN (ACK), eliminating timing guesses

Event-driven startup: wrapper blocks on poll(timeout) and proceeds only after receiving READY + PID

```
// Daemon ends must survive exec()
if (!clearCloExec(sv_startup[1]) || !clearCloExec(sv_shutdown[1])) {
    std::perror( s: "fcntl FD_CLOEXEC");
    close(sv_startup[0]); close(sv_startup[1]);
    close(sv_shutdown[0]); close(sv_shutdown[1]);
    return info;
}
char fdStartupStr[16], fdShutdownStr[16];
std::snprintf(fdStartupStr,  maxlen: sizeof(fdStartupStr),  format: "%d", sv_startup[1]);
std::snprintf(fdShutdownStr,  maxlen: sizeof(fdShutdownStr),  format: "%d", sv_shutdown[1]);
setenv( name: "NOTIFY_FD",  value: fdStartupStr,  replace: 1);
setenv( name: "SHUTDOWN_FD",  value: fdShutdownStr,  replace: 1);
pid_t child = fork();
if (child < 0) {
    std::perror( s: "fork");
    close(sv_startup[0]); close(sv_startup[1]);
    close(sv_shutdown[0]); close(sv_shutdown[1]);
    unsetenv( name: "NOTIFY_FD");
    unsetenv( name: "SHUTDOWN_FD");
    return info;
}
```

Pass the handshake file descriptors through exec() using environment variables (FD_CLOEXEC cleared), then spawn the daemon process.

```
waitpid( pid: child,  stat_loc: nullptr,  options: 0);
// Wait PID from daemon (startup handshake), timeout 5s
pollfd pfd{};
pfd.fd = sv_startup[0];
pfd.events = POLLIN | POLLHUP;
int ret = poll(&pfd,  nfds: 1,  timeout: 5000);
if (ret > 0 && (pfd.revents & (POLLIN | POLLHUP))) {
    info.pid = readPidFromFd(sv_startup[0]);
} else {
    std::cerr << "[Wrapper] ERROR: startup timeout for " << binName << std::endl;
}
close(sv_startup[0]);
if (info.pid <= 0) {
    // startup failed: close shutdown channel too
    close(sv_shutdown[0]);
    info.shutdown_fd = -1;
    info.pid = -1;
    return info;
}
// Keep shutdown read end for later
info.shutdown_fd = sv_shutdown[0];
return info;
```

ESRG

```cpp
static void daemonize(const char* pidfile, const char* logfile = nullptr) {
    pid_t pid = fork();
    if (pid < 0) std::exit( status: EXIT_FAILURE);
    if (pid > 0) std::exit( status: EXIT_SUCCESS);

    if (setsid() < 0) std::exit( status: EXIT_FAILURE);

    std::signal( sig: SIGHUP, SIG_IGN);

    pid = fork();
    if (pid < 0) std::exit( status: EXIT_FAILURE);
    if (pid > 0) std::exit( status: EXIT_SUCCESS);

    umask( mask: 0);
    chdir( path: "/");

    close( fd: STDIN_FILENO);
    open( file: "/dev/null",  oflag: O_RDONLY);

    if (logfile) {
        int fd = open(logfile,  oflag: O_WRONLY | O_CREAT | O_APPEND, 0644);
        if (fd >= 0) {
            dup2(fd,  fd2: STDOUT_FILENO);
            dup2(fd,  fd2: STDERR_FILENO);
            close(fd);
        } else {
            close( fd: STDOUT_FILENO); close( fd: STDERR_FILENO);
            open( file: "/dev/null",  oflag: O_WRONLY);
            open( file: "/dev/null",  oflag: O_WRONLY);
        }
    } else {
        close( fd: STDOUT_FILENO); close( fd: STDERR_FILENO);
        open( file: "/dev/null",  oflag: O_WRONLY);
        open( file: "/dev/null",  oflag: O_WRONLY);
    }

    int fd = open(pidfile,  oflag: O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd >= 0) {
        char buf[32];
        ssize_t n = std::snprintf(buf,  maxlen: sizeof(buf),  format: "%d\n", getpid());
        (void)write(fd, buf, n);
        close(fd);
    }
}
```

Uses a double-fork pattern to detach the service from the terminal and create an independent session

Normalize daemon environment: reset umask and move working directory to /

Redirects output to log files and records the Process ID (PID) for system management.

Startup & Shutdown Daemonization


```
static void stopDaemon(DaemonInfo& daemon) {
    if (daemon.pid <= 0) return;

    std::cout << "[Wrapper] Stopping " << daemon.name << " (PID " << daemon.pid << ")..." << std::endl;

    // Request graceful termination
    if (kill(daemon.pid, sig: SIGTERM) != 0 && errno == ESRCH) {
        if (daemon.shutdown_fd >= 0) { close(daemon.shutdown_fd); daemon.shutdown_fd = -1; }
        return;
    }

    if (daemon.shutdown_fd >= 0) {
        pollfd pfd{};
        pfd.fd = daemon.shutdown_fd;
        pfd.events = POLLIN | POLLHUP;

        int ret = poll(&pfd, nfds: 1, timeout: 5000);
        if (ret > 0 && (pfd.revents & (POLLIN | POLLHUP))) {
            // Accept either explicit "OK" or just EOF
            char buf[16];
            (void)read(daemon.shutdown_fd, buf, nbytes: sizeof(buf));
            close(daemon.shutdown_fd);
            daemon.shutdown_fd = -1;
            std::cout << "[Wrapper] " << daemon.name << " stopped (ACK/EOF)." << std::endl;
            return;
        }

        // Timeout or error
        close(daemon.shutdown_fd);
        daemon.shutdown_fd = -1;
    }

    std::cout << "[Wrapper] " << daemon.name << " did not respond. Forcing SIGKILL." << std::endl;
    kill(daemon.pid, sig: SIGKILL);
}
```

Graceful shutdown request: send SIGTERM to let the daemon clean up

Wait for shutdown acknowledgement with a bounded timeout (poll).

Startup & Shutdown Daemonization


```
static void sendShutdownAck() {
    if (g_shutdown_fd >= 0) {
        const char* ack = "OK\n";
        (void)write(g_shutdown_fd, ack, n:3);
        close(g_shutdown_fd);
        g_shutdown_fd = -1;
    }
}
```

On termination, the daemon confirms cleanup completion by sending an 'OK' acknowledgement and closing the channel

```
// Startup notify (PID or -1)
if (notify_fd >= 0) {
    char buf[64];
    int len = ok ? std::snprintf(buf, maxlen: sizeof(buf), format: "%d\n", getpid())
                 : std::snprintf(buf, maxlen: sizeof(buf), format: "-1\n");
    (void)write(notify_fd, buf, len);
    close(notify_fd);
    notify_fd = -1;
}
```

After initialization succeeds, the daemon notifies the wrapper with its PID (or -1 on failure)